Dynamic Analysis - Fuzzing IoT Devices

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

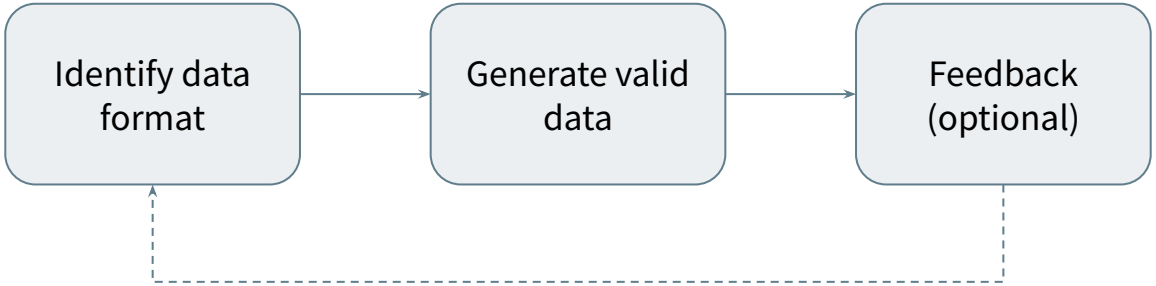
Fuzzing IoT Devices

- Input Generation
- Execution Control
- Invalid State Detection

Input Generation: Challenges

- Complex formats:
  - Belkin Wemo: SOAP+XML over HTTP
- Stateful Inputs:
  - Philips Hue: authenticated commands
    - Need to authenticate first and then send commands.
- Proprietary formats:
  - LIFX Mini C (custom byte/value formats)

Input Generation



Input Generation: Identifying Input Formats

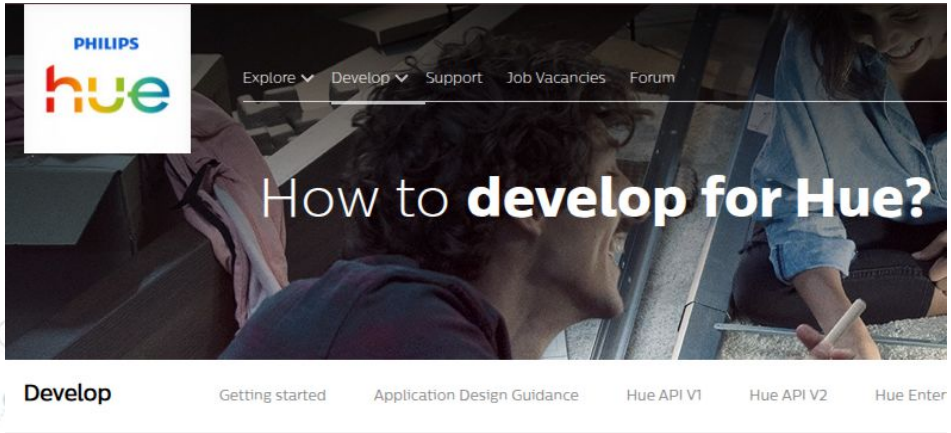
Input Generation: Identifying Input Formats

- Extracting from documentation: Standard APIs, vendor websites, etc.
- Traffic Analysis: Record the network traffic and infer.
- Firmware/App reverse engineering: Analyze the app/firmware to identify message structure.
- Active Probing: Modify messages and learn the fields.

Identifying Input Formats: Extracting from documentation

Vendors publish their APIs for compatibility and extensibility.

E.g., Philips Hue Bridge




```
PUT /lights/1/state

{
  "on": true,
  "sat": 254,
  "bri": 254,
  "hue": 10000
}
```

Identifying Input Formats: Traffic Analysis

Record traffic to/from IoT devices and analyze the traffic to infer formats.

Action in app	Captured packet (hex)	Length (bytes)
Turn ON	`AA 55 01 02 10 00 01 19`	8
Turn OFF	`AA 55 01 02 10 00 00 18`	8
Set level = 100	`AA 55 01 02 10 00 64 7C`	8
Set level = 50	`AA 55 01 02 10 00 32 4A`	8
Set level = 0	`AA 55 01 02 10 00 00 18`	8

Identifying Input Formats: Traffic Analysis

Align and analyze:

Action	B0	B1	B2	B3	B4	B5	B6	B7
Turn ON	AA	55	01	02	10	00	01	19
Turn OFF	AA	55	01	02	10	00	00	18
Level = 100	AA	55	01	02	10	00	64	7C
Level = 50	AA	55	01	02	10	00	32	4A
Level = 0	AA	55	01	02	10	00	00	18

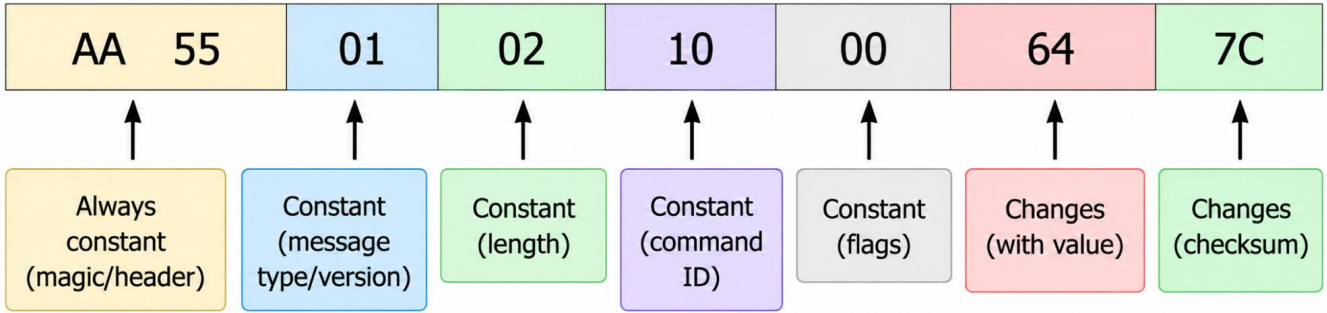
Identifying Input Formats: Traffic Analysis

Align and analyze:

Action	B0	B1	B2	B3	B4	B5	B6	B7
Turn ON	AA	55	01	02	10	00	01	19
Turn OFF	AA	55	01	02	10	00	00	18
Level = 100	AA	55	01	02	10	00	64	7C
Level = 50	AA	55	01	02	10	00	32	4A
Level = 0	AA	55	01	02	10	00	00	18

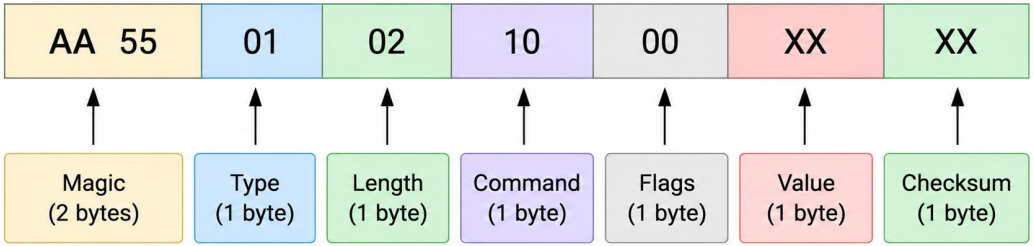
Identifying Input Formats: Traffic Analysis

Identify constants and variable fields:



Identifying Input Formats: Traffic Analysis

Infer message format:



[Header]	[Type]	[Len]	[Cmd]	[Flags]	[Value]	[CRC]
2 bytes	1 byte	1 byte	1 byte	1 byte	1 byte	1 byte

Identifying Input Formats: Traffic Analysis

Test your hypothesis:

Turn ON	Value = 1	AA 55 01 02 10 00 01 19
Turn OFF	Value = 0	AA 55 01 02 10 00 00 18
Set level = 25	Value = 25	AA 55 01 02 10 00 19 35
Set level = 100	Value = 100	AA 55 01 02 10 00 64 7C

Identifying Input Formats: Firmware/App analysis

Analyze the firmware/controller (companion app) to identify the message structure.

Identifying Input Formats: Firmware/App analysis

Analyze the firmware/controller (companion app) to identify the message structure.

- Start with known send/receive commands and analyze back from there.

IoTFuzzer (NDSS 2018) and DIANE (S&P 2021) hooks sender methods in companion apps and mutates the arguments.

Identifying Input Formats: Firmware/App analysis

Where to hook?

set_temparature(t) -> send_temp_command(t) -> add_checksum(data) -> sock.send(data)

Identifying Input Formats: Firmware/App analysis

Where to hook?

```
set_temparature(t) {
  if (t>100 || t<0) return -1;
  send_temp_command(t);
}
```

Identifying Input Formats: Firmware/App analysis

Where to hook?

```
send_temp_command(t) {
  d = add_header();
  encode_temp(d, t);
  add_checksum(d);
 sock.send(d);
}
```

Identifying Input Formats: Firmware/App analysis

Where to hook?

We need maximal control and minimal chance of generating invalid inputs.

Identifying Input Formats: Active Probing

Modify messages and figure out:

Message: {"on":true}

"on":true} → invalid JSON

{on":true} → invalid JSON

{"n":true} → unknown parameter "n"

{"o":true} → unknown parameter "o"

{"on:true} → invalid JSON

{"on":true → invalid JSON

Identifying Input Formats: Active Probing

Modify messages and figure out:

"on":true} → invalid JSON

{on":true} → invalid JSON

{"n":true} → unknown parameter "n"

{"o":true} → unknown parameter "o"

{"on:true} → invalid JSON

{"on":true → invalid JSON

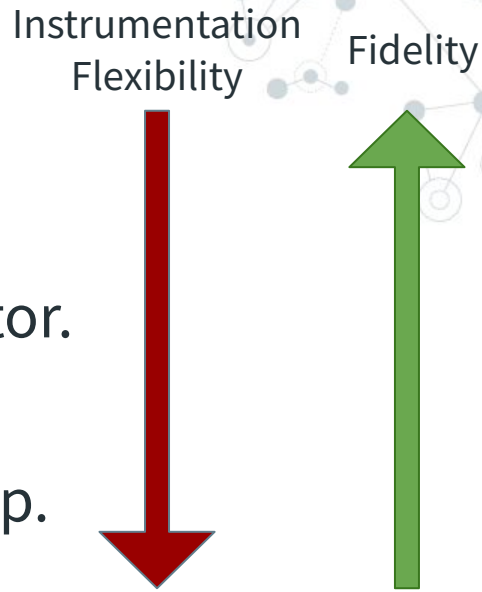
https://dl.acm.org/doi/10.1145/3460120.3484543 (SnipFuzz)

Execution Control

- On Device.
- On Simulator: High-fidelity device simulator.
- Hybrid: Host and Device; Hardware-in-loop.
- Rehosting: Virtualized environment. Generic simulator + device models.

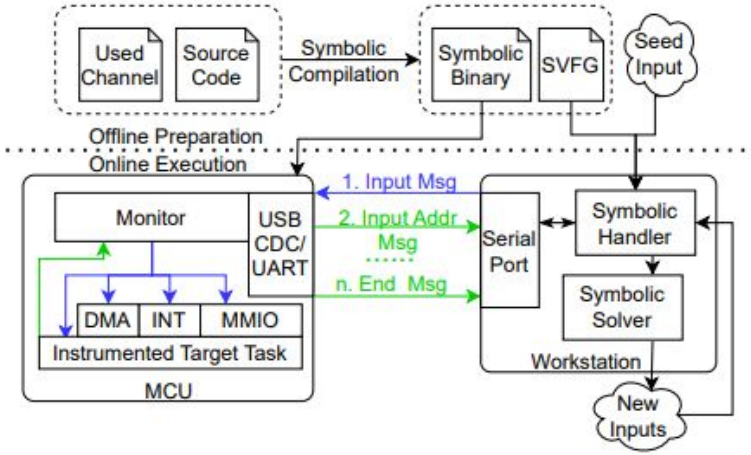
Execution Control

- On Device.
- On Simulator: High-fidelity device simulator.
- Hybrid: Host and Device; Hardware-in-loop.
- Rehosting: Virtualized environment. Generic simulator + device models.



Execution Control: Hardware in loop

Use hardware for execution and offload heavy weight instrumentation to host machine.



Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf



Concolic Execution or Dynamic Symbolic Execution (DSE)

- Random testing or fuzzing cannot generate highly constraint inputs.

- Can we use symbolic execution to generate these constrained inputs?

DSE Approach

1. Start with random input values.

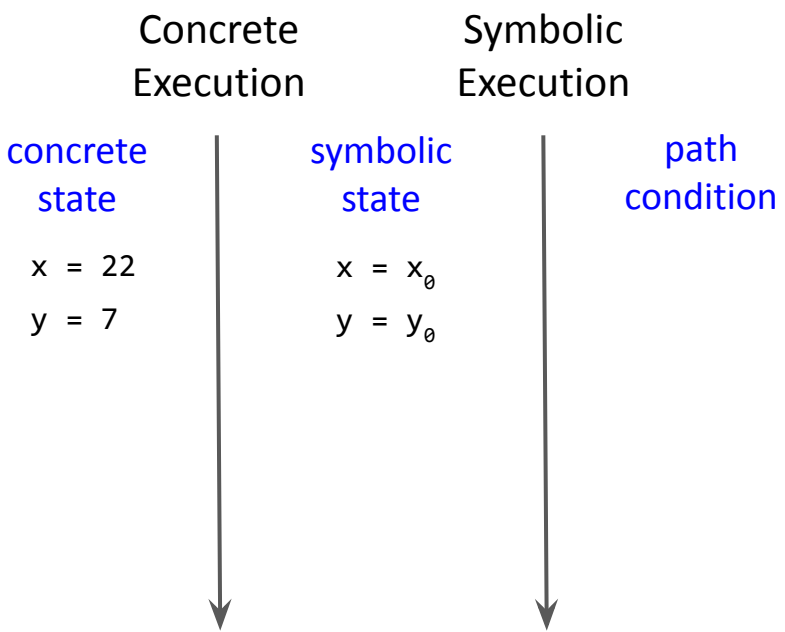
2. Keep track of both concrete values and symbolic constraints.

3. Use concrete values to simplify symbolic constraints.

DSE Example

```
int foo(int v) {
   return 2*v;
}

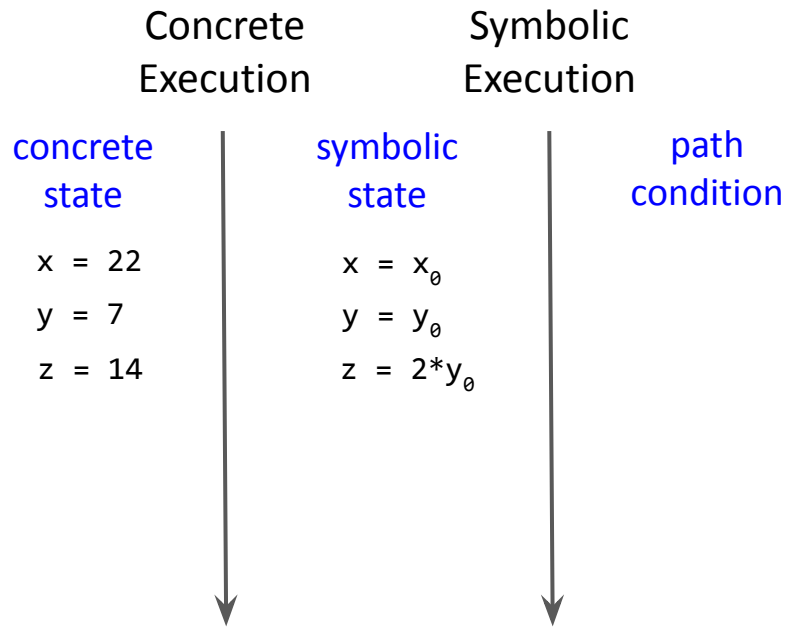
void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```

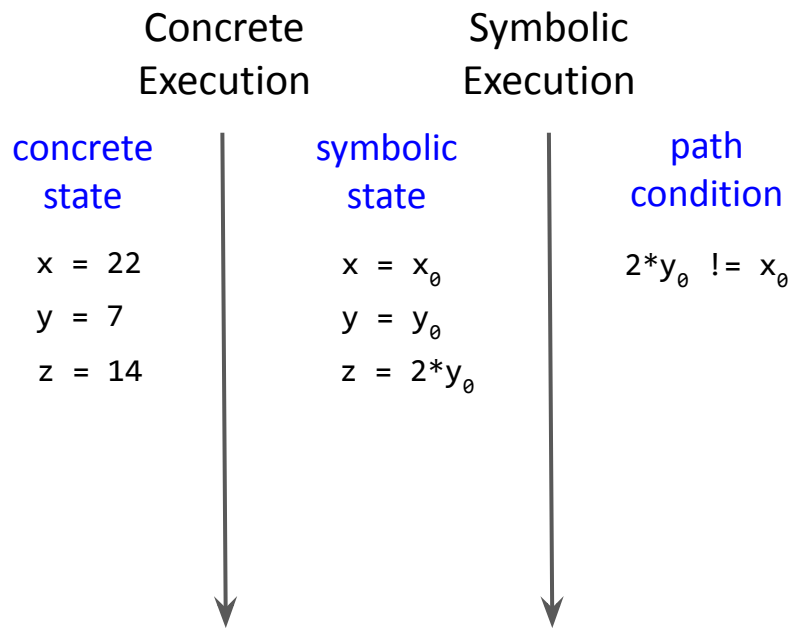


DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```





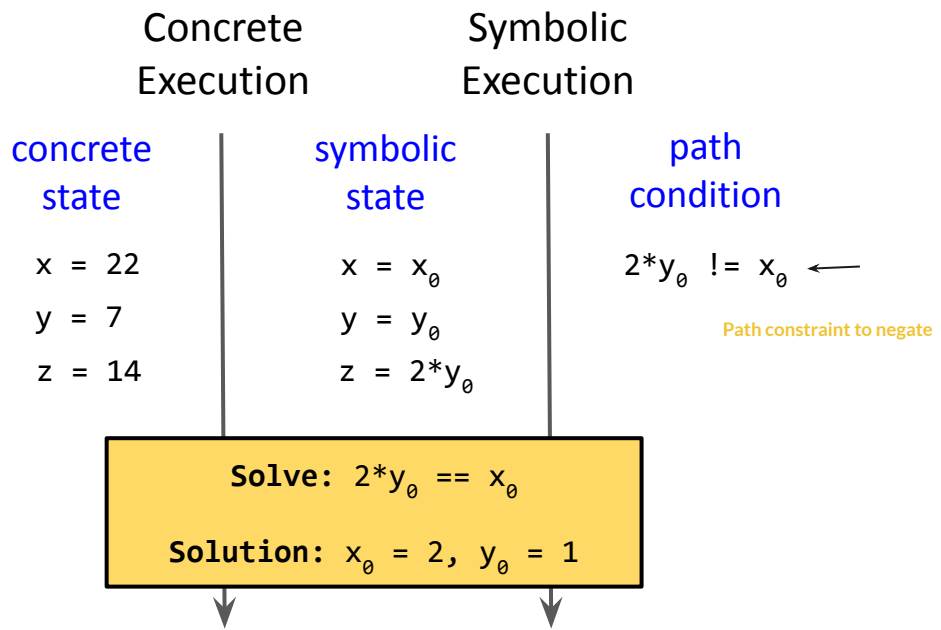
DSE Example

Now to generate new input => Negate the path condition and solve to get inputs that will make the program execute another path.

DSE Example

```
int foo(int v) {
   return 2*v;
}

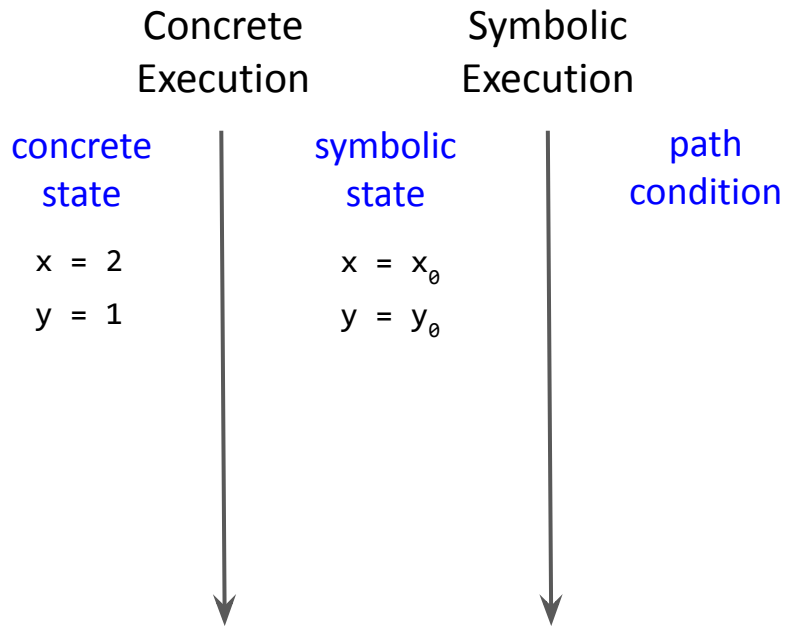
void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```

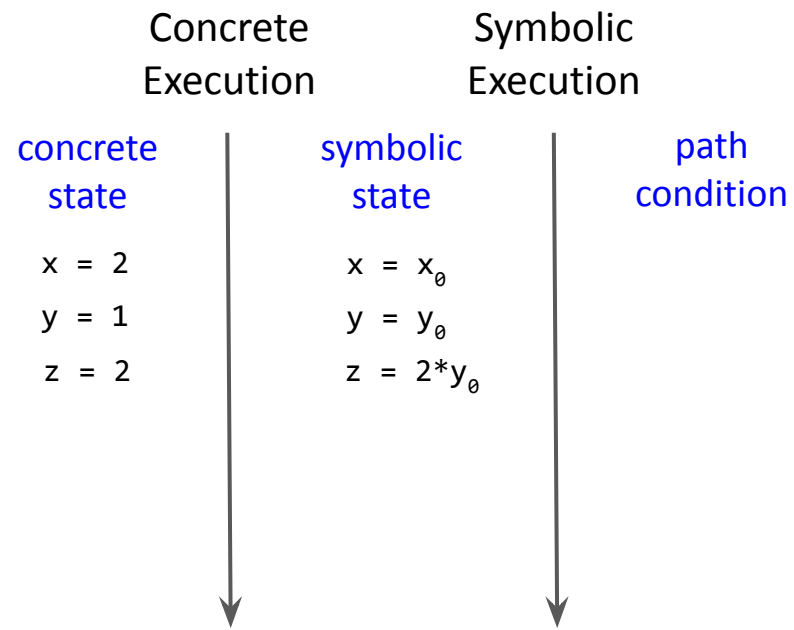


DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



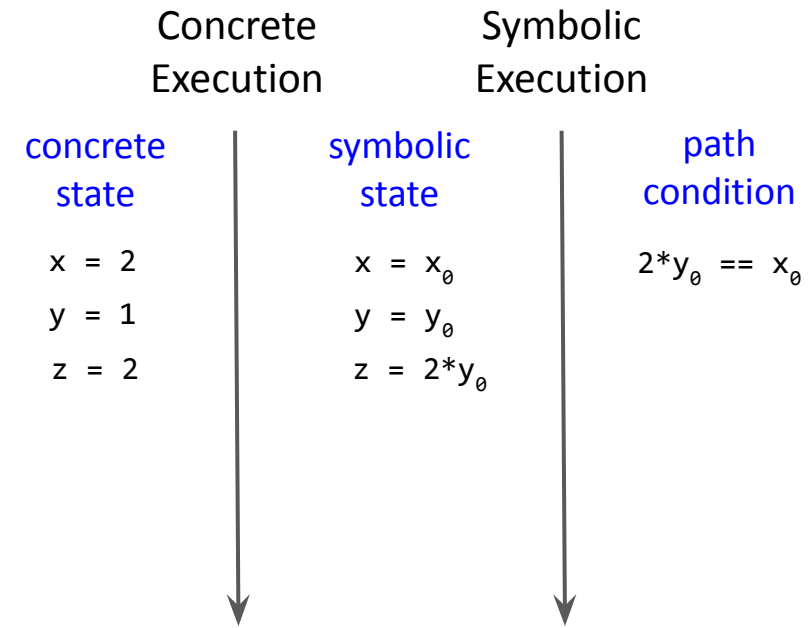


DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



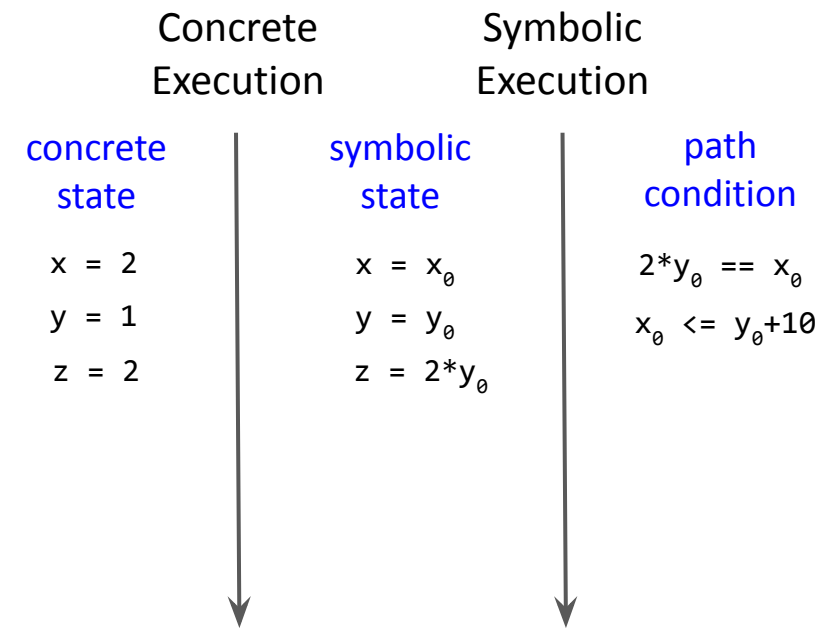


DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```

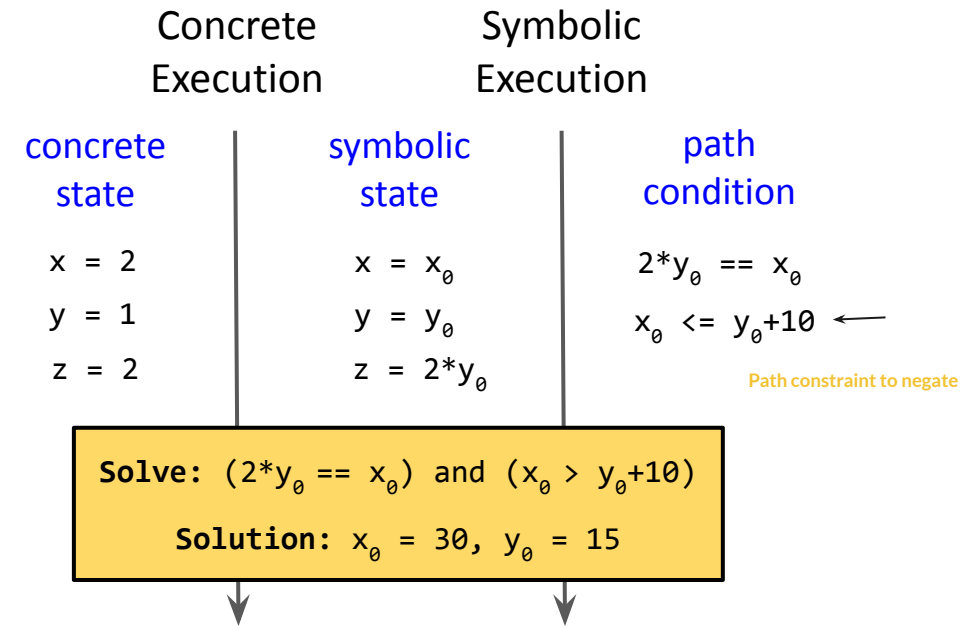




DSE Example

```
int foo(int v) {
   return 2*v;
}

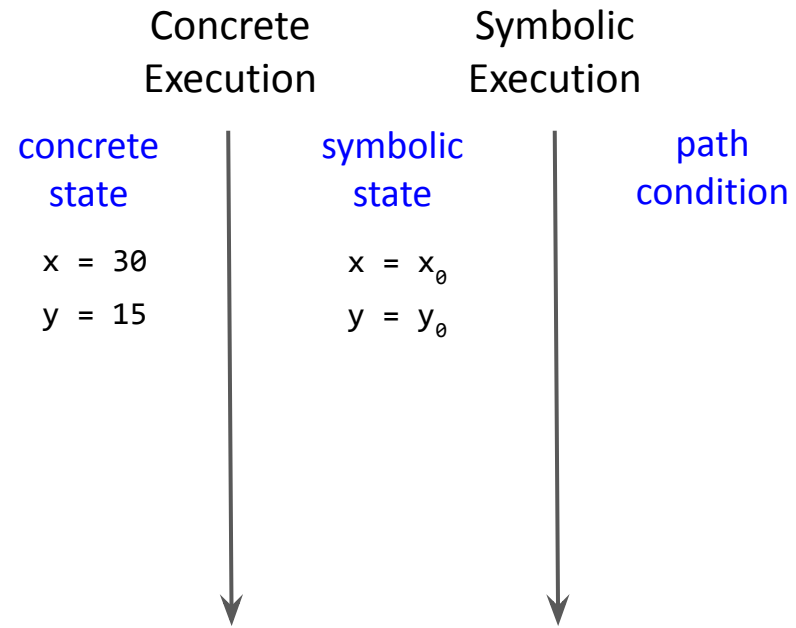
void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



DSE Example

```
int foo(int v) {
   return 2*v;
}

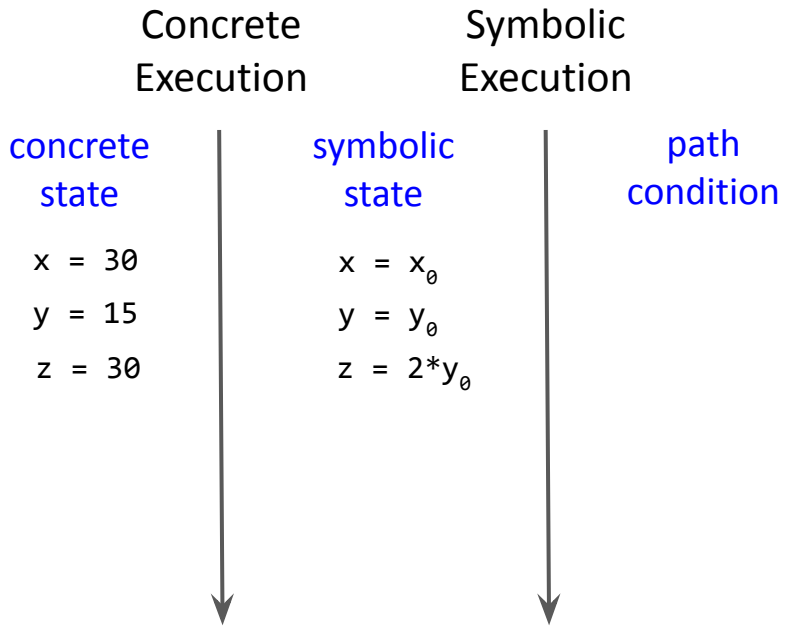
void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



DSE Example

```
int foo(int v) {
   return 2*v;
}

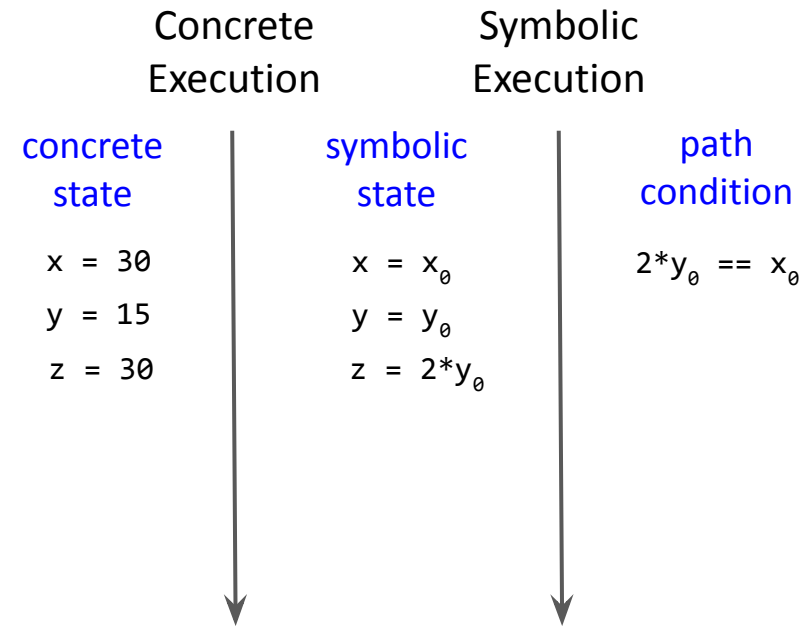
void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



DSE Example

```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



DSE Example

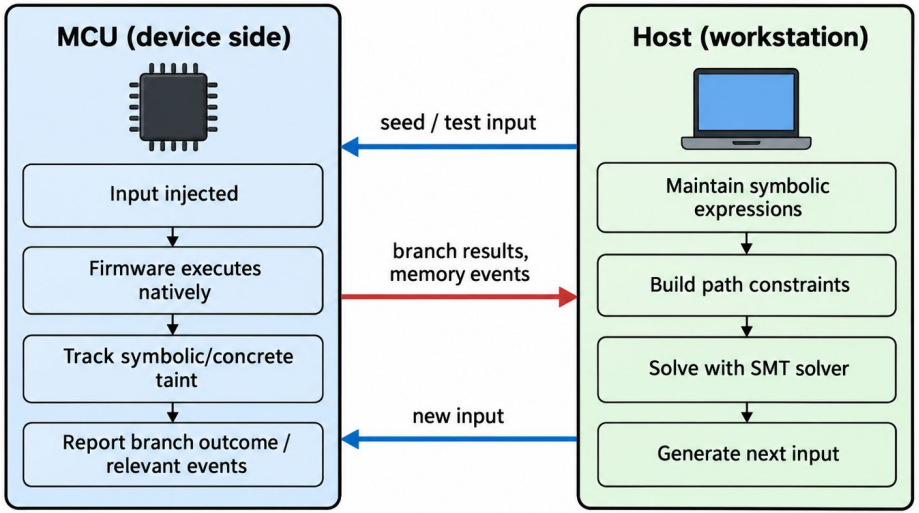
```
int foo(int v) {
   return 2*v;
}

void test_me(int x, int y) {
   int z = foo(y);
   if (z == x)
      if (x > y+10)
         ERROR;
}
```



Concrete Execution	Symbolic Execution	
concrete state	symbolic state	path condition
x = 30	x = x_0	$2*y_0 == x_0$
y = 15	y = y_0	$x_0 > y_0+10$
z = 30	z = $2*y_0$	

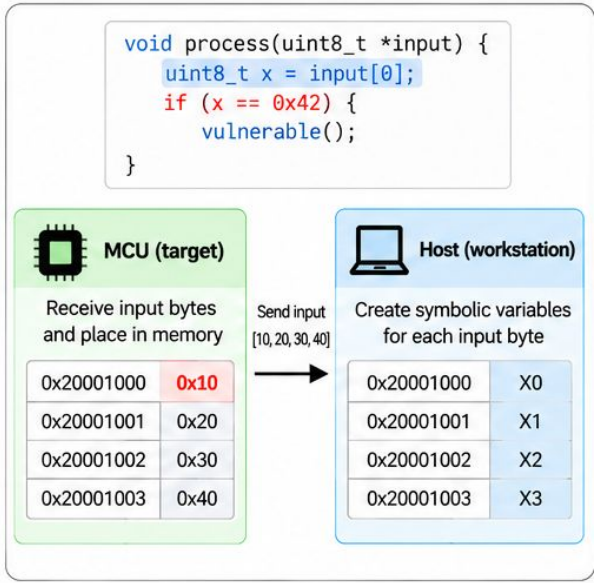
Execution Control: Hybrid (Concolic Co-execution)



MCU runs the firmware concretely; the host performs symbolic reasoning.

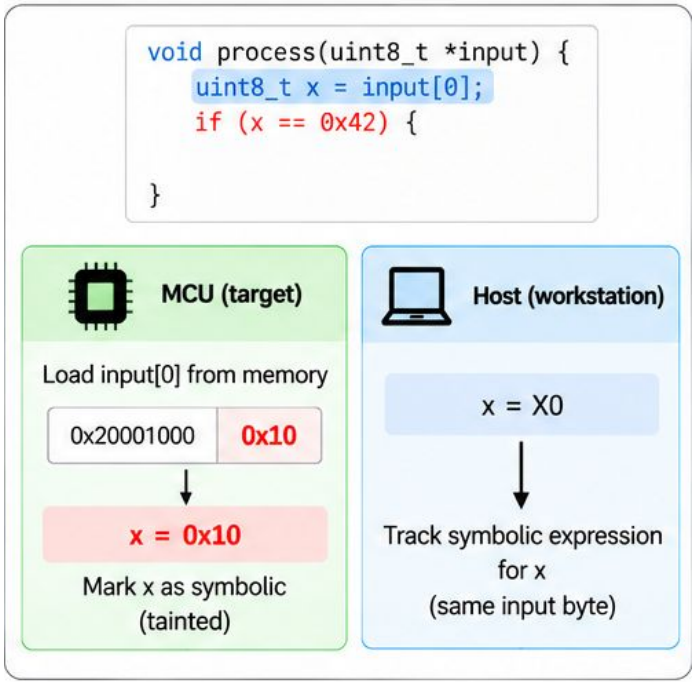
Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

Concolic Co-execution



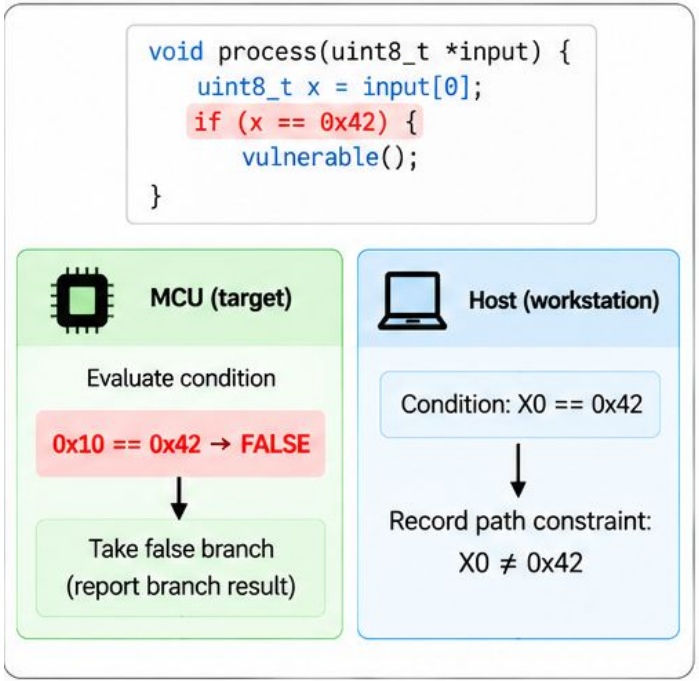
Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

Concolic Co-execution



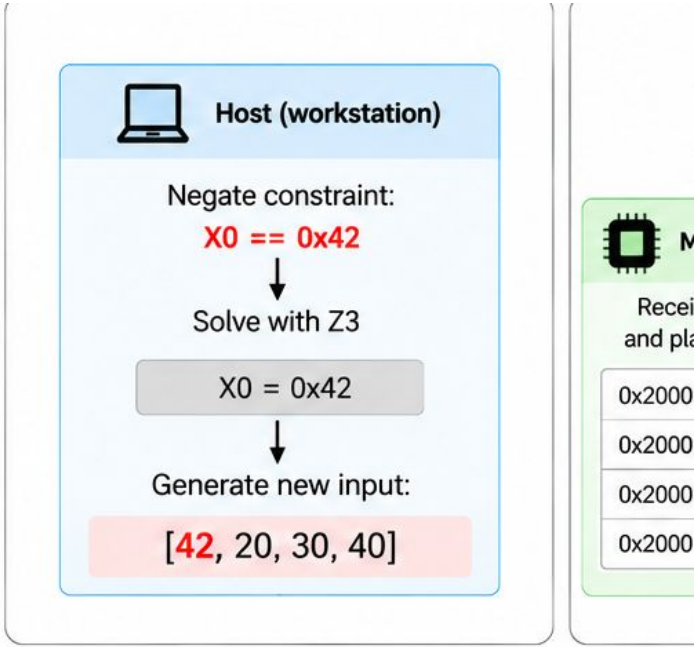
Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

Concolic Co-execution



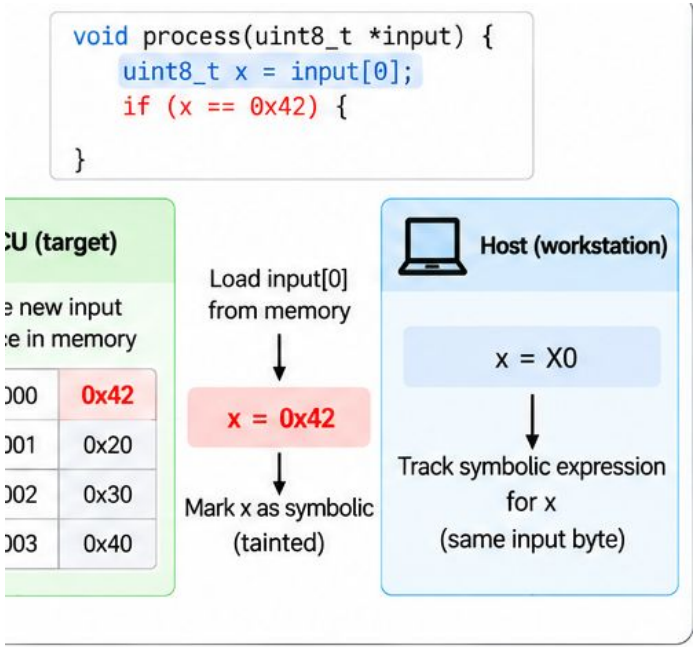
Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

Concolic Co-execution



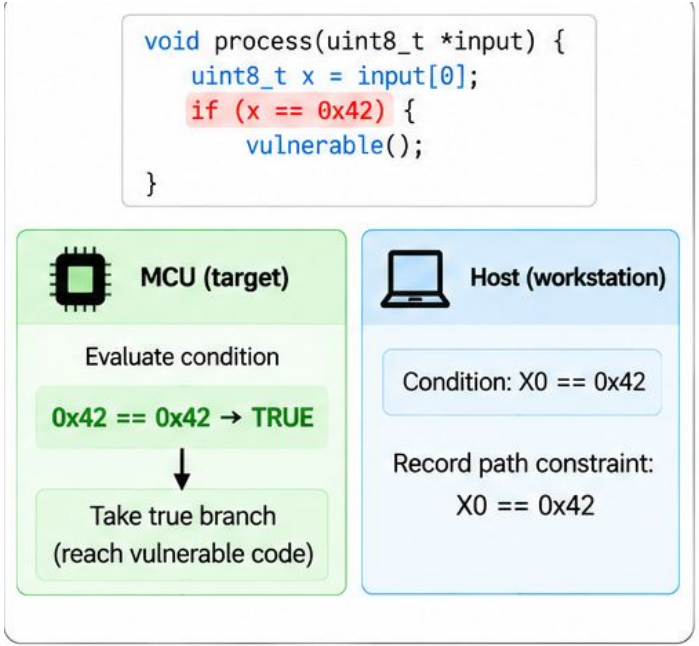
Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

Concolic Co-execution



Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

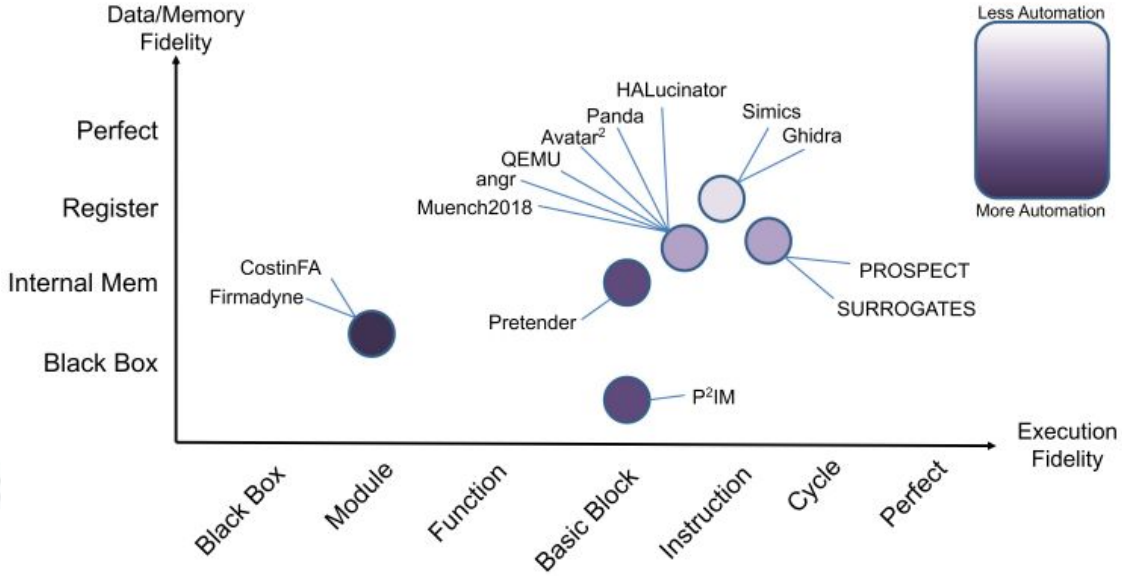
Concolic Co-execution



Concolic co execution: https://www.usenix.org/system/files/usenixsecurity24-liu-changming.pdf

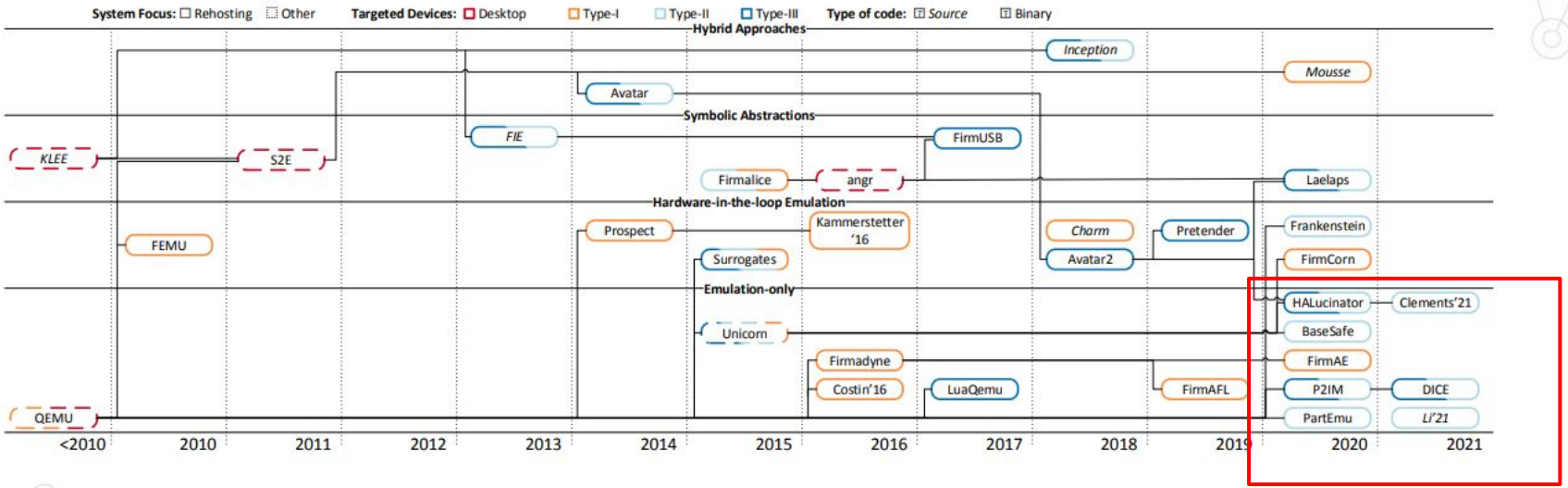
Execution Control: Rehosting

Executing the firmware in virtual environment.



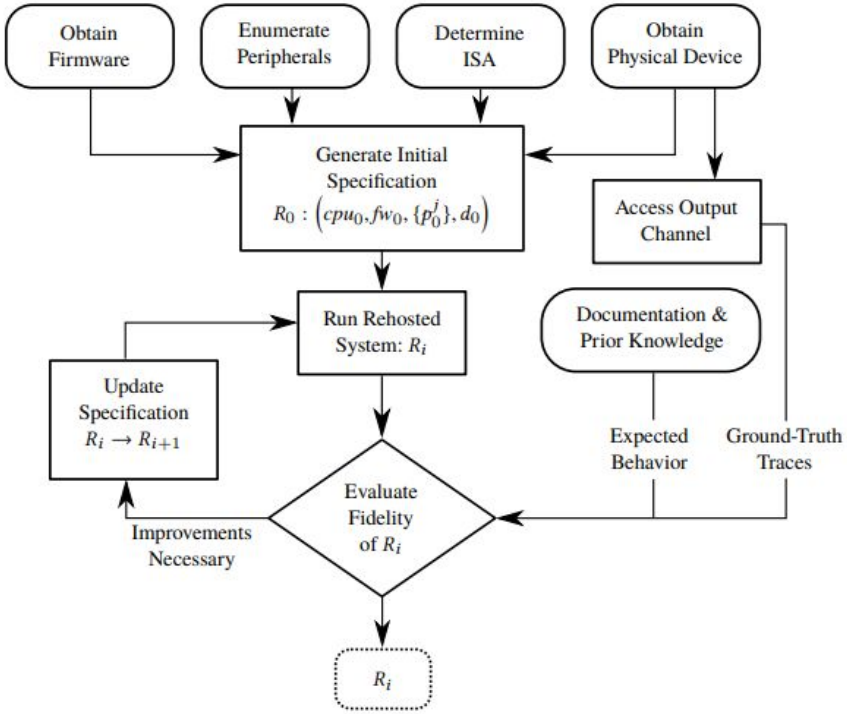
https://dl.acm.org/doi/pdf/10.1145/3423167

Rehosting: Trend (Emulation only preferred)



SoK Enabling Security Analysis: https://dl.acm.org/doi/pdf/10.1145/3433210.3453093

Rehosting an embedded system



SoK Enabling Security Analysis: https://dl.acm.org/doi/pdf/10.1145/3433210.3453093

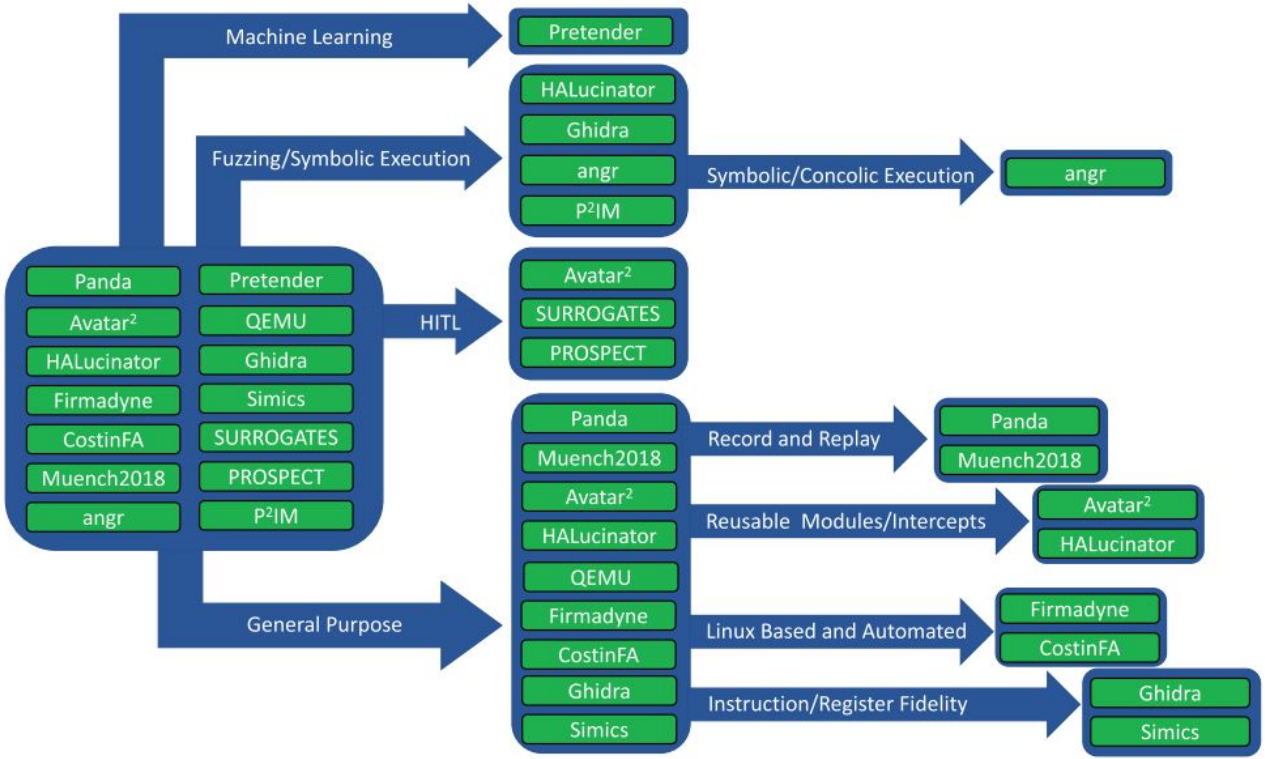
Rehosting

Need emulation environment.

Paper/Technique	External Hardware and Peripherals	Mem Interactions/ Setup	Configure Hardware	Missing Code	Function Identification and Labeling
angr [151]	✓	✓			✓
Avatar² [112]	✓	✓	✓	✓	✓
CostinFA [43, 44]	✓	✓	✓	✓	✓
Firmadyne [25]	–	✓	✓	–	✓
Ghidra [119]	✓	✓			✓
HALucinator [35]	✓	✓	✓	✓	✓
Muench2018 [113]	✓	✓	✓	✓	✓
P²IM [57]	✓	✓	✓	✓	
PANDA [50]	✓	✓	✓	✓	
Pretender [72]	✓	✓	✓	✓	✓
PROSPECT [84]	✓	✓	✓	✓	
QEMU [11]	✓	✓	✓	✓	
Simics [105]	✓	✓	✓	✓	
SURROGATES [90]	✓	✓	✓	✓	
ARMX [146]	✓	✓	✓	✓	✓
Datasheets	✓	✓			
Firmalice [150]	✓	✓			✓
ICSREF [86]					✓
IDA-PRO [136]	✓	✓			✓
Inception [40]	✓	✓			✓
KLEE [19]	✓	✓			✓
OFFDTAN [177]		✓			
PIE: Parser [36]	✓	✓			✓
Radare2 [164]	✓	✓			✓
Spedi [87]	✓	✓			✓

https://dl.acm.org/doi/pdf/10.1145/3423167

Rehosting: Choosing emulation environment



https://dl.acm.org/doi/pdf/10.1145/3423167



A significant challenge to re-hosting firmware is the large expanse of hardware peripherals that need to be emulated.

WRIGHT et.al

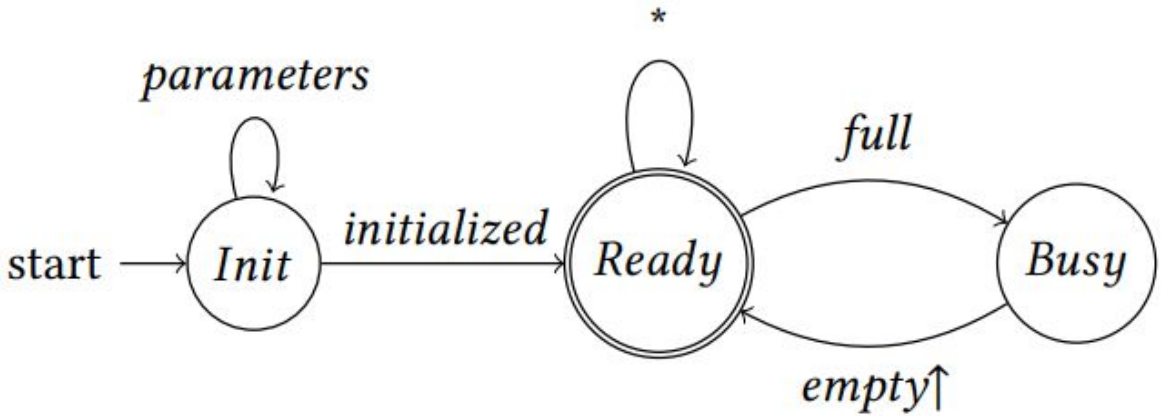
Challenges in Firmware Re-Hosting, Emulation, and Analysis

PURDUE UNIVERSITY

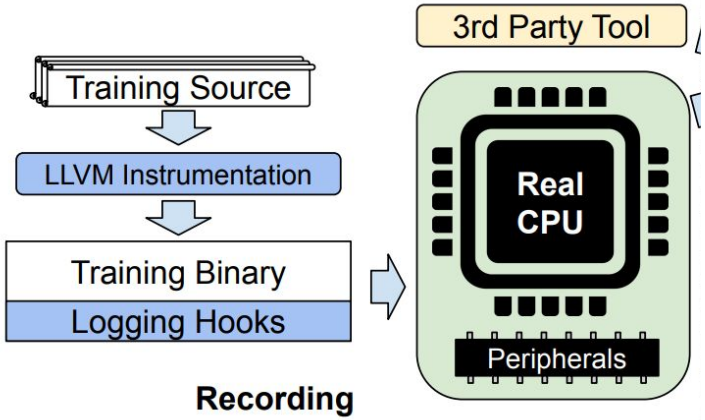
Modeling Peripherals

- Manual:
  - HALucinator (write abstractions for HAL layer)
- Automated:
  - Static analysis.
  - Record and learn: on-the-fly (Fuzzware) or offline (Conware).
- Hybrid:
  - P2IM: Manually create abstract models and instantiate them to real device.

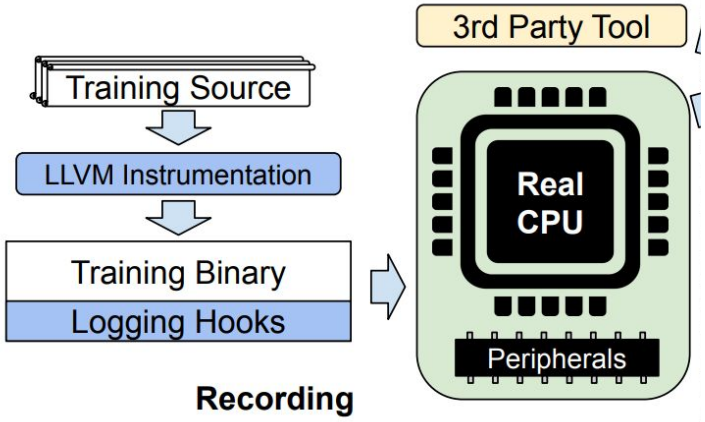
Conware: Record and Learn



Conware: Record and Learn

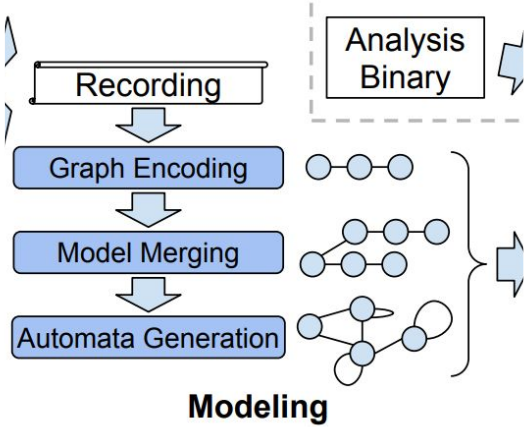


Conware: Record and Learn

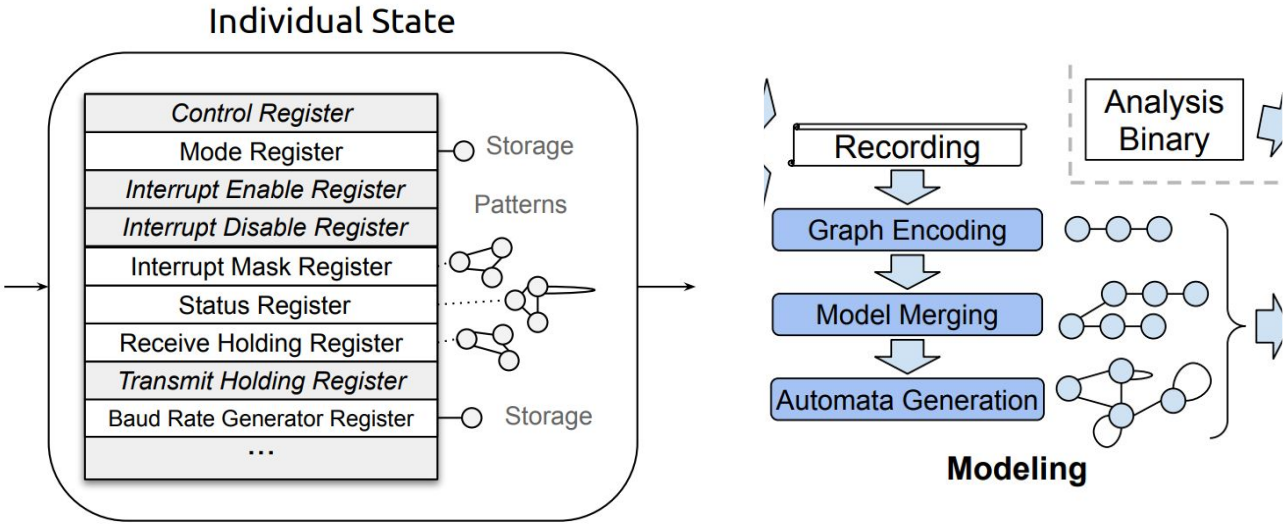


Operation	Address	Value
⋮	⋮	⋮
WRITE	0x400E0800 (control)	0x50
READ	0x400E0814 (status)	0x40001A1A (ready)
WRITE	0x400E081C (TX)	0x4F | **O**
READ	0x400E0814 (status)	0x40001818 (busy)
READ	*(repeats 434×)*	*(repeats 434×)*
READ	0x400E0814 (status)	0x4000181A (ready)
WRITE	0x400E081C (TX)	0x4E | **N**
READ	0x400E0814 (status)	0x40001818 (busy)
READ	*(repeats 2,634×)*	*(repeats 2,634×)*
READ	0x400E0814 (status)	0x4000181A (ready)
⋮	⋮	⋮

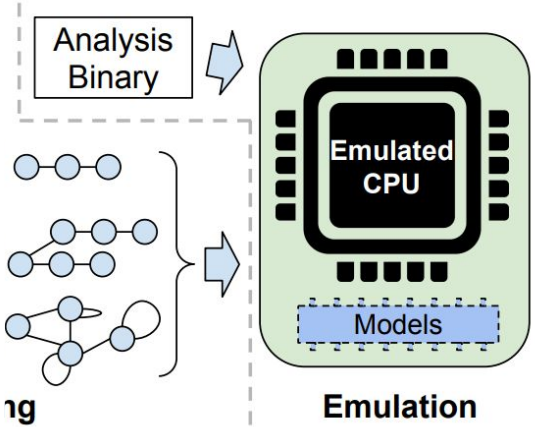
Conware: Record and Learn



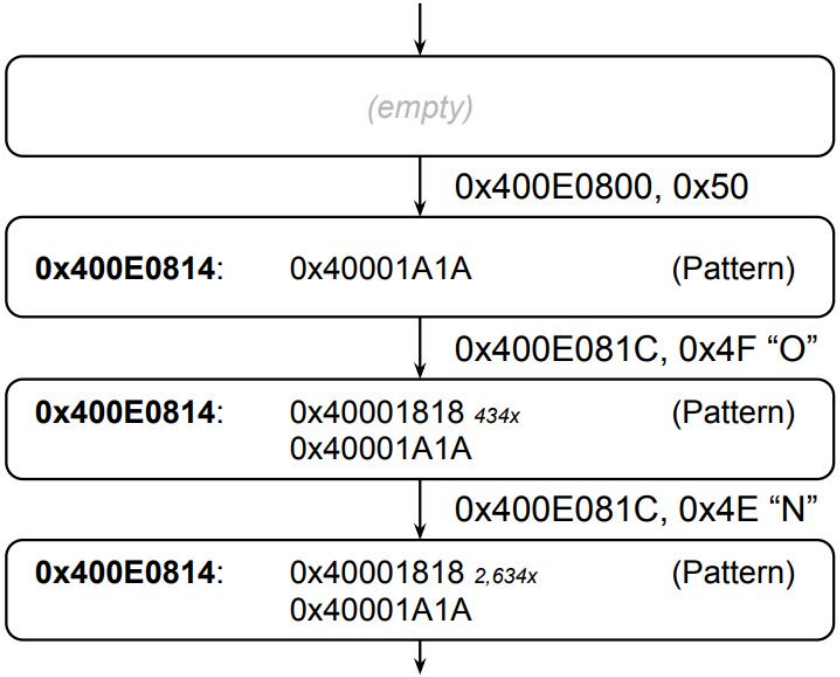
Conware: Record and Learn

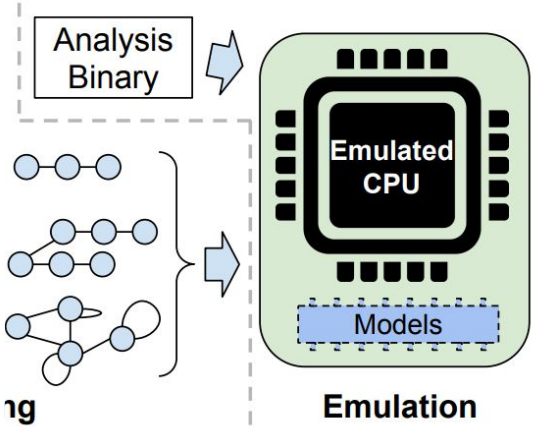


Conware: Record and Learn

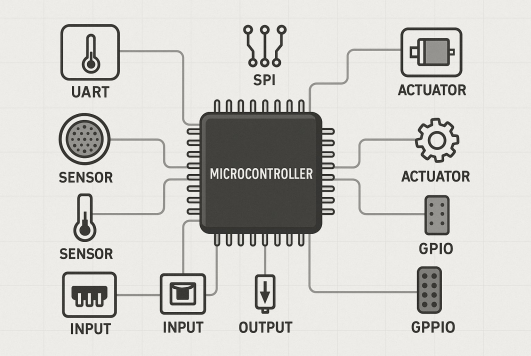


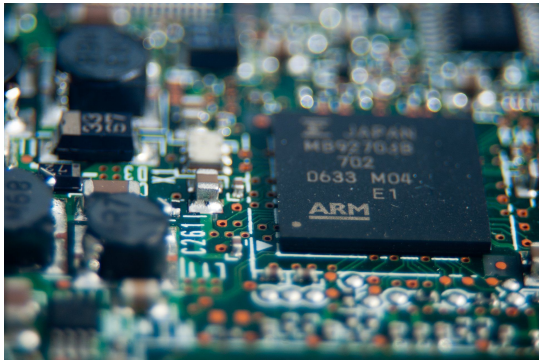
Conware: Record and Learn

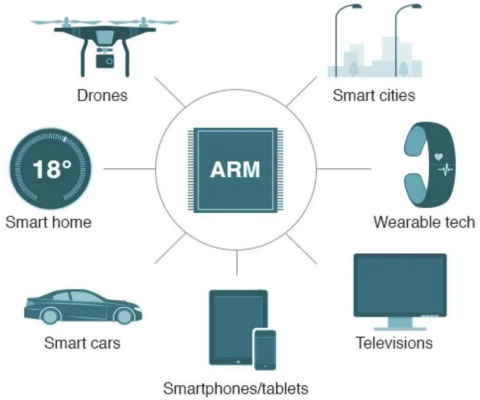




Scalability Issues in peripheral modeling Due to Diversity







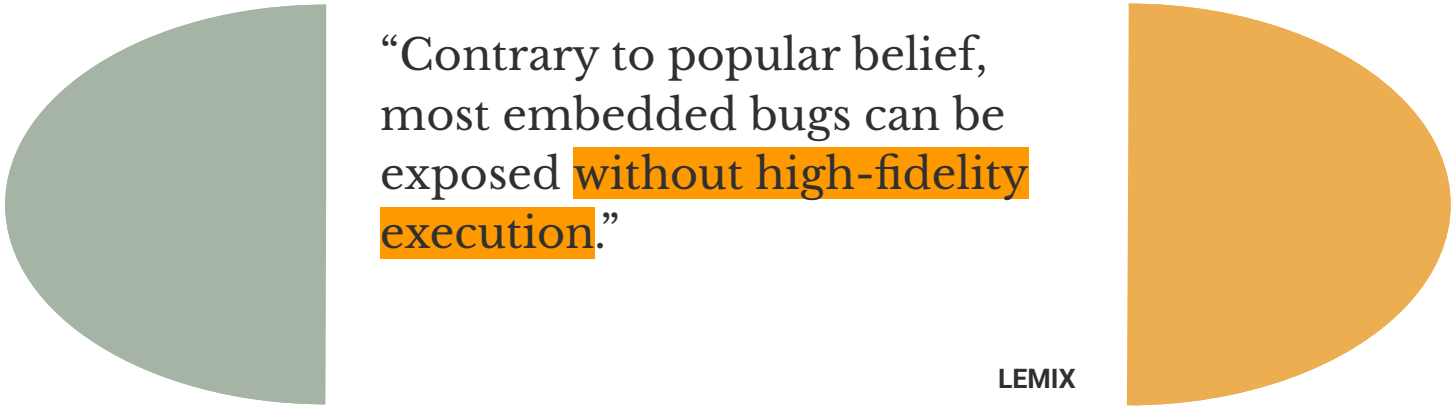
NXP

MIPS TECHNOLOGIES

TEXAS INSTRUMENTS

PURDUE UNIVERSITY

Our Hypothesis



What is Fidelity?

Extrapolating on Previous work, we define **Execution Fidelity** as a 4-tuple:

EF = ⟨ S, A, P, C ⟩
where each dimension reflects:

- **S**: Language Semantic Fidelity
- **A**: Assembly Execution Fidelity
- **P**: Peripheral Handling Fidelity
- **C**: Clock Fidelity

Challenges in Firmware Re-Hosting, Emulation, and Analysis

CHRISTOPHER WRIGHT, Purdue University
WILLIAM A. MOEGLEIN, Sandia National Laboratories
SAURABH BAGCHI, Purdue University
MILIND KULKARNI, Purdue University
ABRAHAM A. CLEMENTS, Sandia National Laboratories

PURDUE UNIVERSITY

What is Fidelity?

Extrapolating on Previous work, we define **Execution Fidelity** as a 4-tuple:

EF = ⟨ S, A, P, C ⟩
where each dimension reflects:

- **S**: Language Semantic Fidelity
- **A**: Assembly Execution Fidelity
- **P**: Peripheral Handling Fidelity
- **C**: Clock Fidelity

Challenges in Firmware Re-Hosting, Emulation, and Analysis

CHRISTOPHER WRIGHT, Purdue University
WILLIAM A. MOEGLEIN, Sandia National Laboratories
SAURABH BAGCHI, Purdue University
MILIND KULKARNI, Purdue University
ABRAHAM A. CLEMENTS, Sandia National Laboratories

Each component is **coarsely categorized** as:
🔵 **Low (L)** 🟡 **Medium (M)** 🟢 **High (H)**

PURDUE UNIVERSITY

Empirical Study

“”

Our analysis of the Memory Corruption CVEs from the Rust4Embedded survey indicates that

60 out of 71 (85%) require low-fidelity execution.

LEMIX

Motivating Example

```
1   int32_t tud_msc_read10_cb(uint32_t lba, uint32_t offset,
2   void* buffer, uint32_t bufsize)
3   {
4     // out of ramdisk
5     if ( lba >= DISK_BLOCK_NUM ) {
6       return -1;
7     }
8     /* Attacker can offset to sensitive memory */
9     uint8_t const* addr = msc_disk[lba] + offset;
10    /* Controlled write to known memory
11        may cause undefined behavior */
12    memcpy(buffer, addr, bufsize);
13    return (int32_t) bufsize;
14  }
```

Listing 1: Lack of bounds check on offset in `tud_msc_read10_cb` allows out-of-bounds read from `msc_disk[lba]`, potentially leading to information disclosure or undefined behavior.

Motivating Example

```
1  int32_t tud_msc_read10_cb(uint32_t lba, uint32_t offset,
2  void* buffer, uint32_t bufsize)
3  {
4    // out of ramdisk
5    if ( lba >= DISK_BLOCK_NUM ) {
6      return -1;
7    }
8    /* Attacker can offset to sensitive memory */
9    uint8_t const* addr = msc_disk[lba] + offset;
10   /* Controlled write to known memory
11       may cause undefined behavior */
12   memcpy(buffer, addr, bufsize);
13   return (int32_t) bufsize;
14 }
```

Listing 1: Lack of bounds check on offset in `tud_msc_read10_cb` allows out-of-bounds read from `msc_disk[lba]`, potentially leading to information disclosure or undefined behavior.

Need to ensure task containing this function gets executed.

Motivating Example

```
1   int32_t tud_msc_read10_cb(uint32_t lba, uint32_t offset,
2   void* buffer, uint32_t bufsize)
3   {
4     // out of ramdisk
5     if ( lba >= DISK_BLOCK_NUM ) {
6       return -1;
7     }
8     /* Attacker can offset to sensitive memory */
9     uint8_t const* addr = msc_disk[lba] + offset;
10    /* Controlled write to known memory
11        may cause undefined behavior */
12    memcpy(buffer, addr, bufsize);
13    return (int32_t) bufsize;
14  }
```

Listing 1: Lack of bounds check on offset in `tud_msc_read10_cb` allows out-of-bounds read from `msc_disk[lba]`, potentially leading to information disclosure or undefined behavior.

Also need to provide valid inputs to the arguments to trigger the bug.

PURDUE UNIVERSITY

Motivating Example

```
1   int32_t tud_msc_read10_cb(uint32_t lba, uint32_t offset,
2   void* buffer, uint32_t bufsize)
3   {
4     // out of ramdisk
5     if ( lba >= DISK_BLOCK_NUM ) {
6       return -1;
7     }
8     /* Attacker can offset to sensitive memory */
9     uint8_t const* addr = msc_disk[lba] + offset;
10    /* Controlled write to known memory
11        may cause undefined behavior */
12    memcpy(buffer, addr, bufsize);
13    return (int32_t) bufsize;
14  }
```

Some way to recognize the security violation.

Listing 1: Lack of bounds check on offset in `tud_msc_read10_cb` allows out-of-bounds read from `msc_disk[lba]`, potentially leading to information disclosure or undefined behavior.

Bug Manifestation Fidelity (BMF)

"Minimum Fidelity required to reach and observe effects of the bugs of interest."

PURDUE UNIVERSITY

Bug Manifestation Fidelity (BMF)

"Minimum Fidelity required to reach and observe effects of the bugs of interest."

Memory Corruption:

- **S**: Language Semantic Fidelity = **High**
- **A**: Assembly Execution Fidelity = **Low**
- **P**: Peripheral Handling Fidelity = **Medium**
- **C**: Clock Fidelity = **Medium**

PURDUE UNIVERSITY

BMF Changes Rehosting Requirements

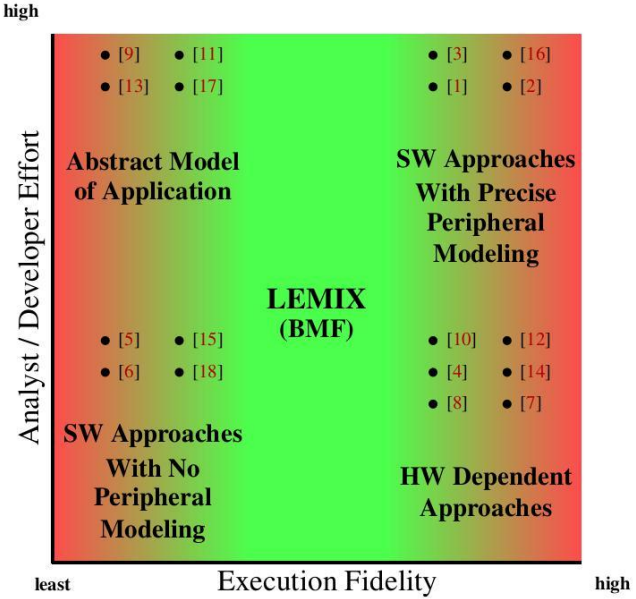


Figure 1: LEMIX in Contrast to Existing Rehosting Approaches.

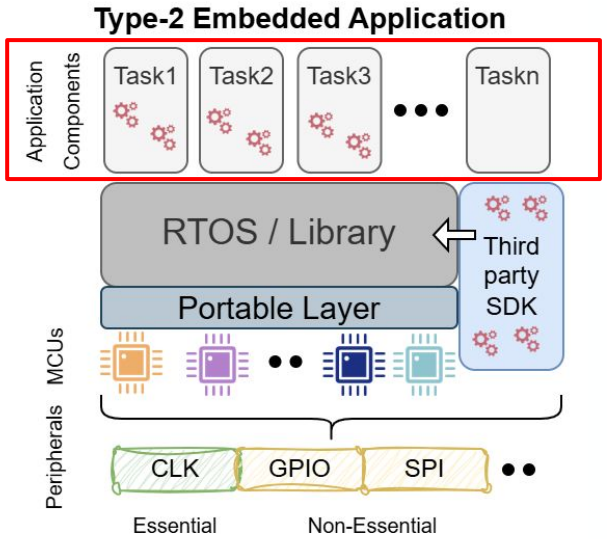
PURDUE UNIVERSITY

Proposed Solution

LEMIX

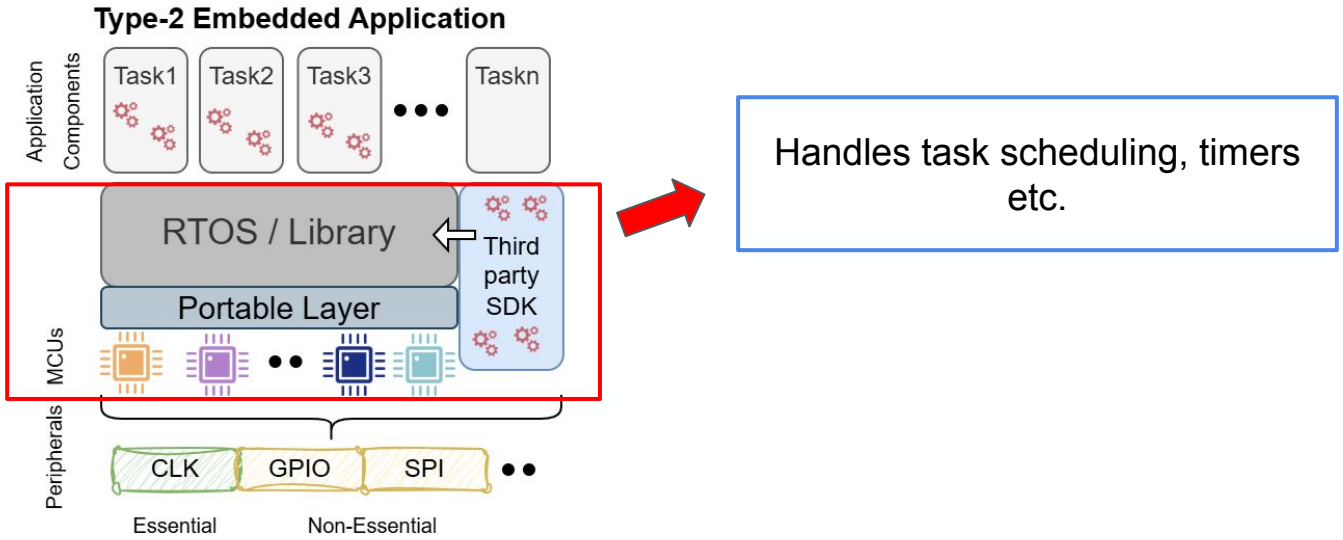
PURDUE UNIVERSITY

LEMIX : Overview



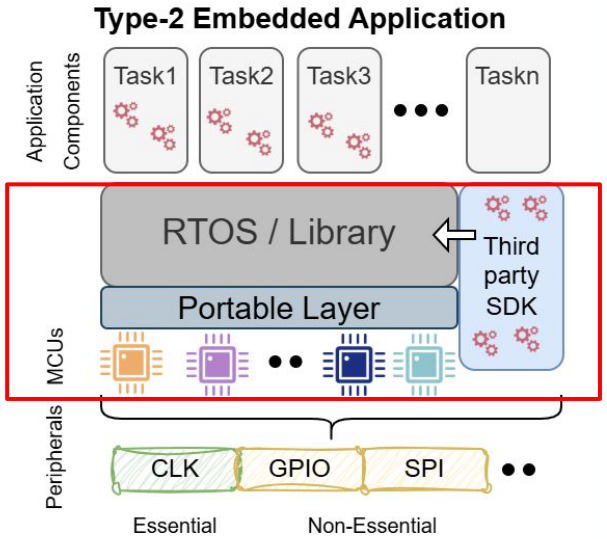
Multiple tasks run simultaneously with priority-driven execution.

LEMIX : Overview



Handles task scheduling, timers etc.

LEMIX : Overview

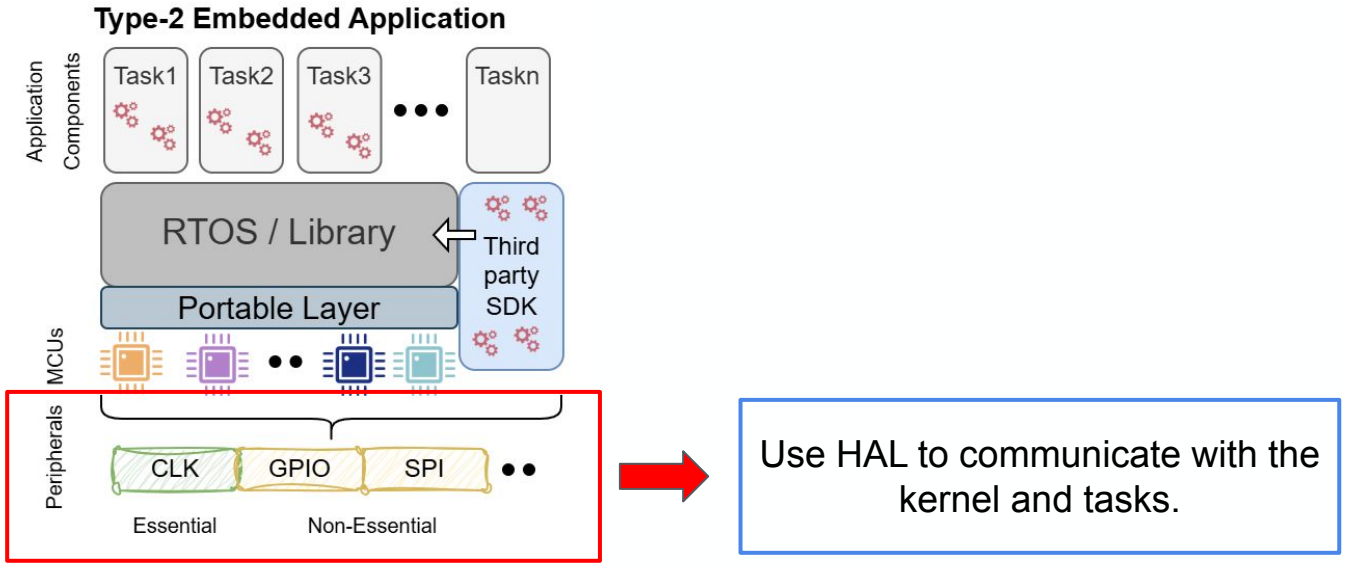


Handles task scheduling, timers etc.

Enables RTOS kernel to run on diverse MCUs.

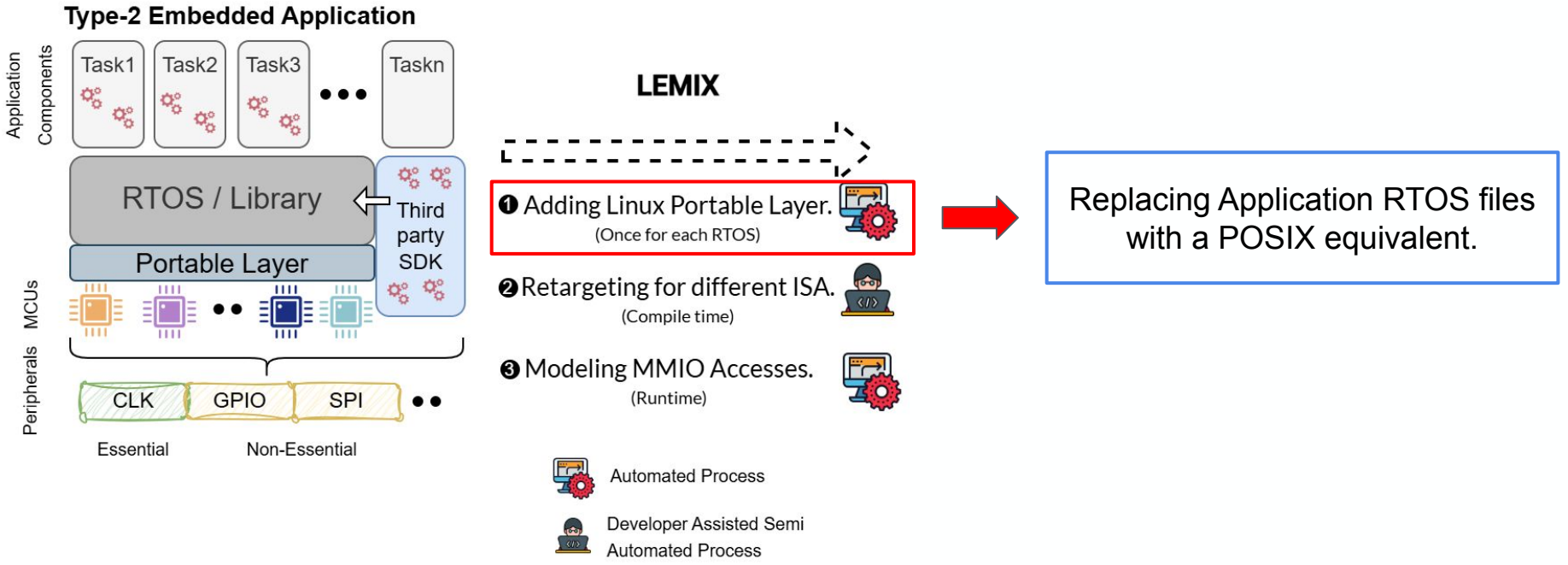
PURDUE UNIVERSITY

LEMIX : Overview

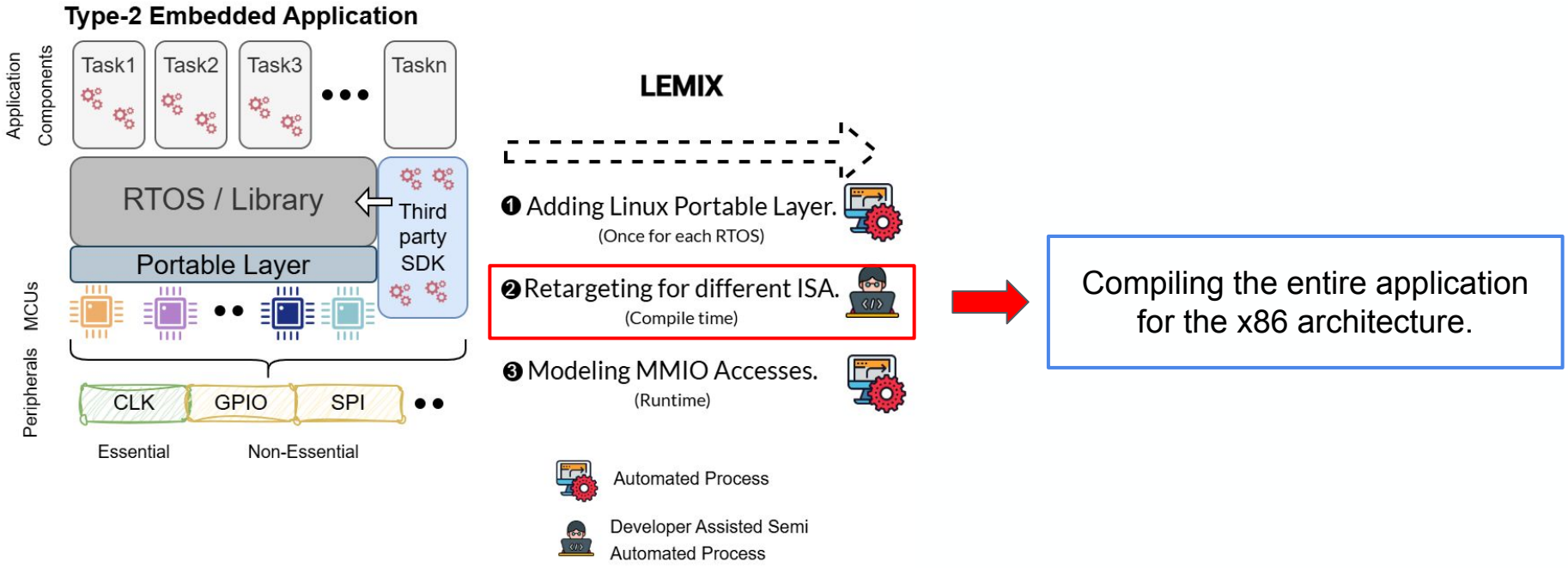


Use HAL to communicate with the kernel and tasks.

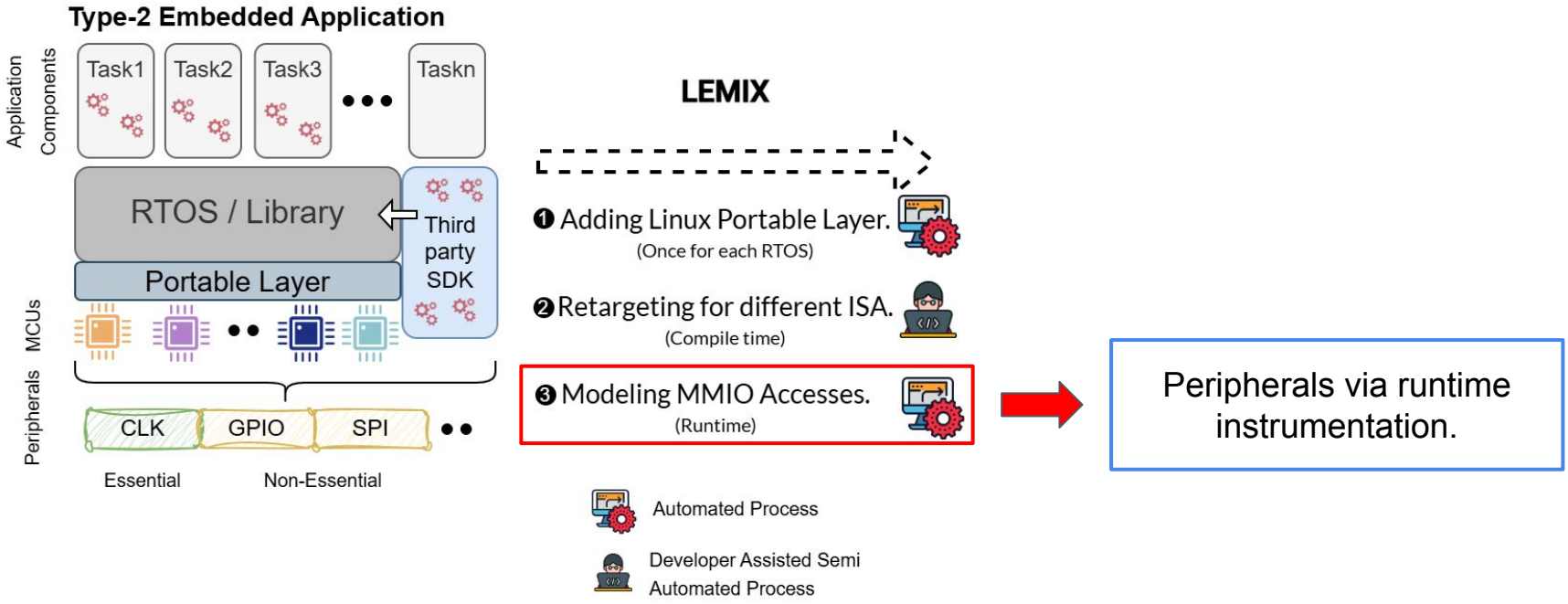
LEMIX : Overview



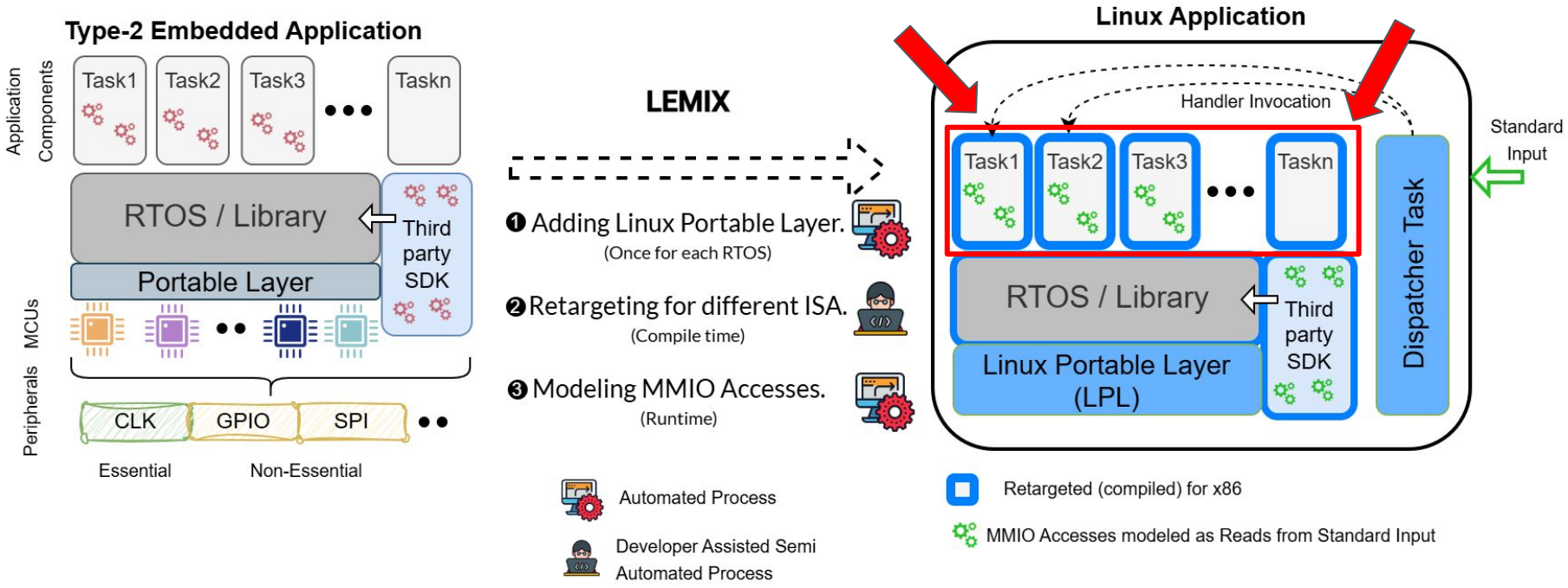
LEMIX : Overview



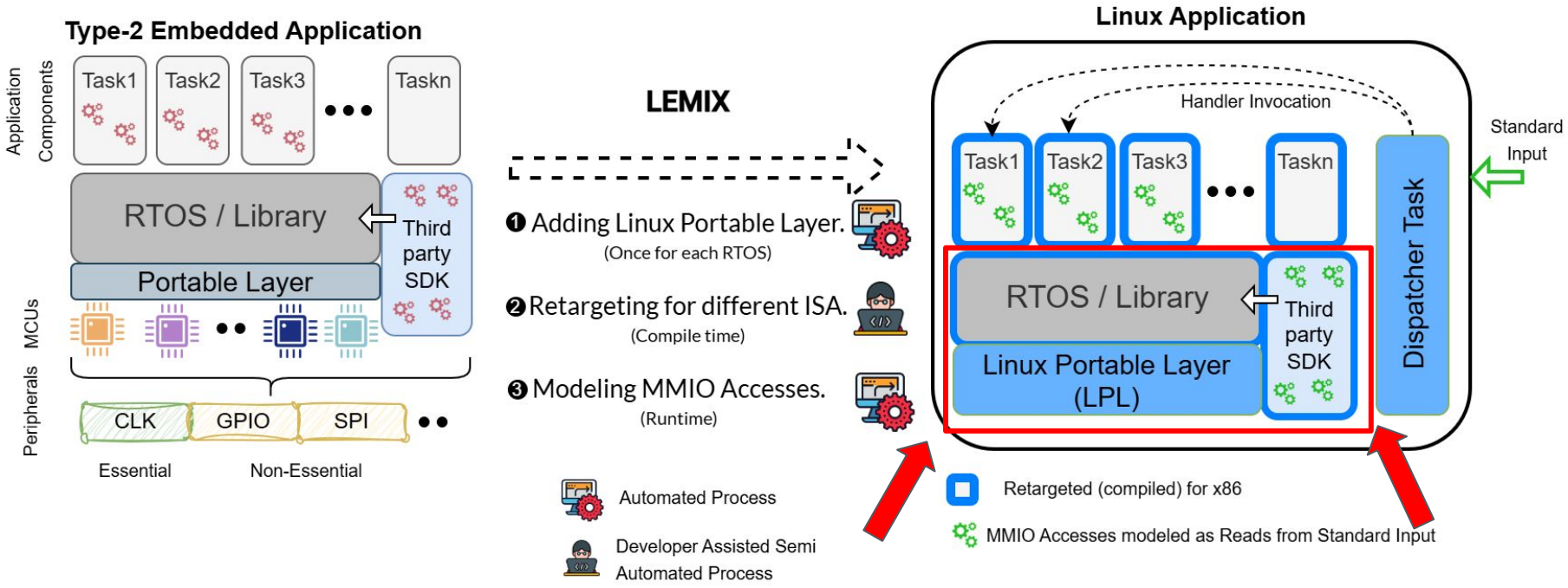
LEMIX : Overview



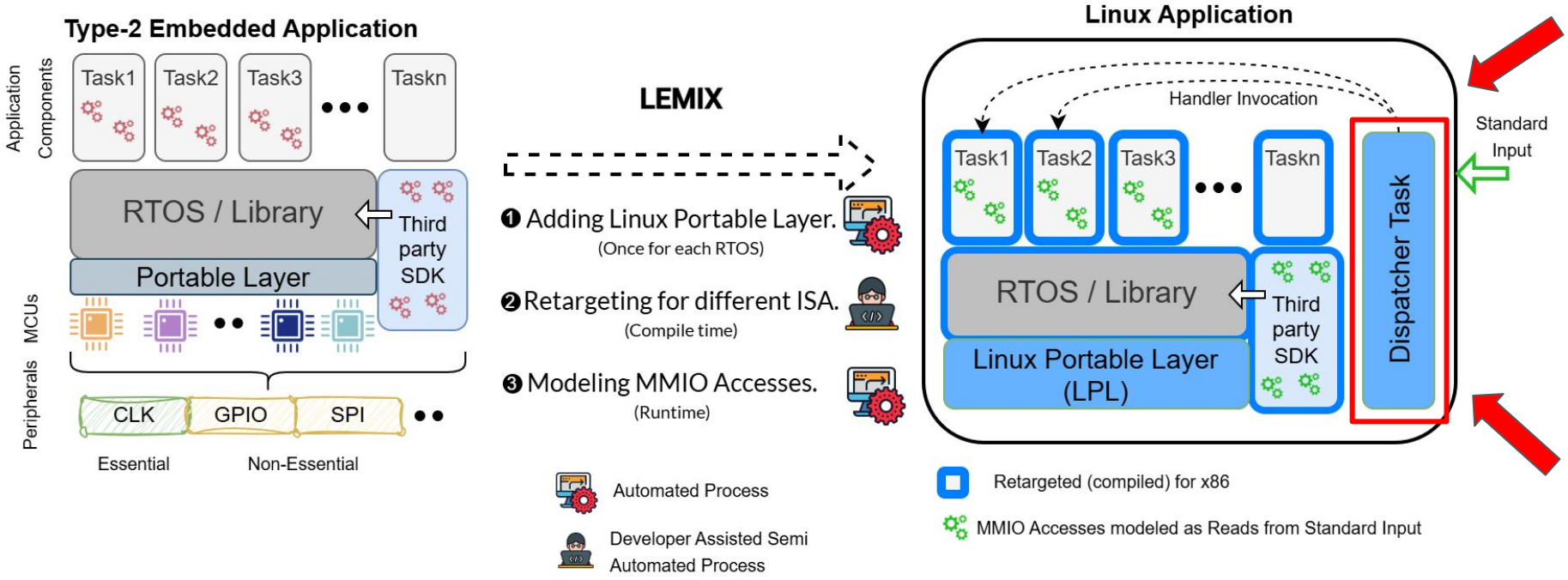
LEMIX : Overview



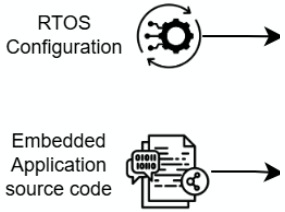
LEMIX : Overview



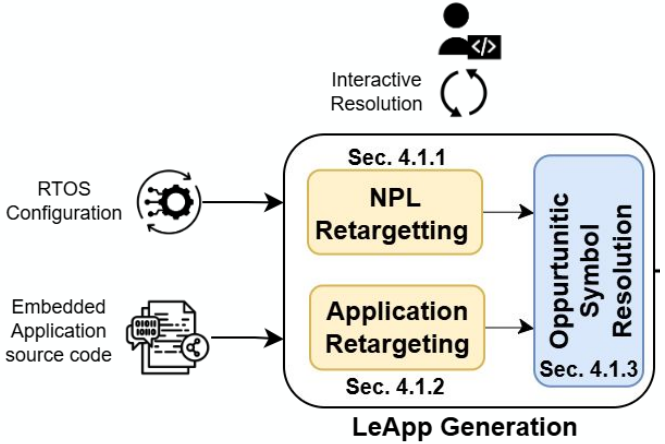
LEMIX : Overview



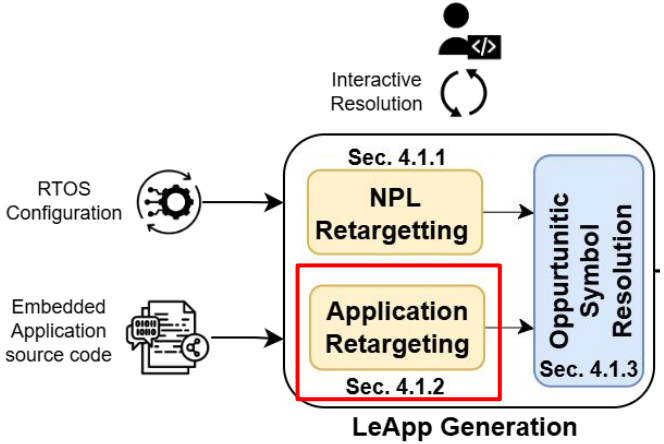
LEMIX : Framework Architecture



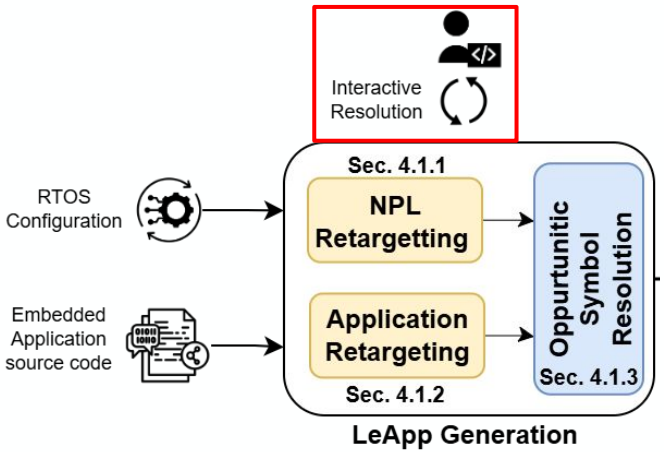
LEMIX : Framework Architecture



LEMIX : Framework Architecture



LEMIX : Framework Architecture

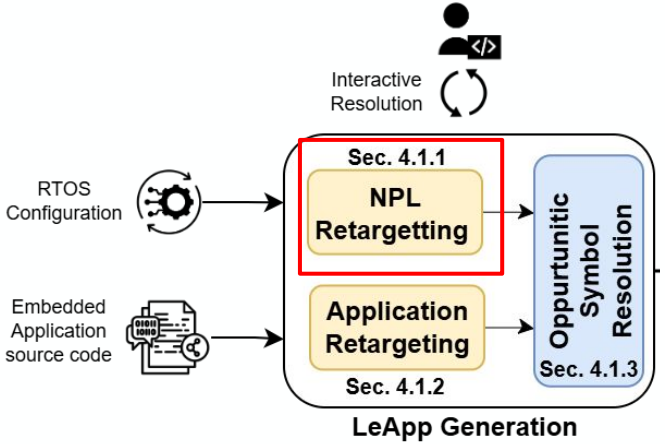



```
1   Effected Source file
2   struct foo {
3         int x;
4        int y[];
5   };
6   struct foo bar = {1, {2, 3, 4}};
7
8   Compilation Fails
9   error: initialization of flexible array member is not allowed
10  struct foo bar = {1, {2, 3, 4}};
11
12  Lemix Provides the Following Guidance Instruction
13
14  1. Search for the definition of the structure which has the
15      flexible member.
16  2. The member declaration will have a [], add a constant
17      value ex. [100] to the member.
18  3. Re-run the framework.
```

Listing 1: shows a sample code which causes compilation error indicated by . provides instructions to the developer to assist in fixing errors shown by .

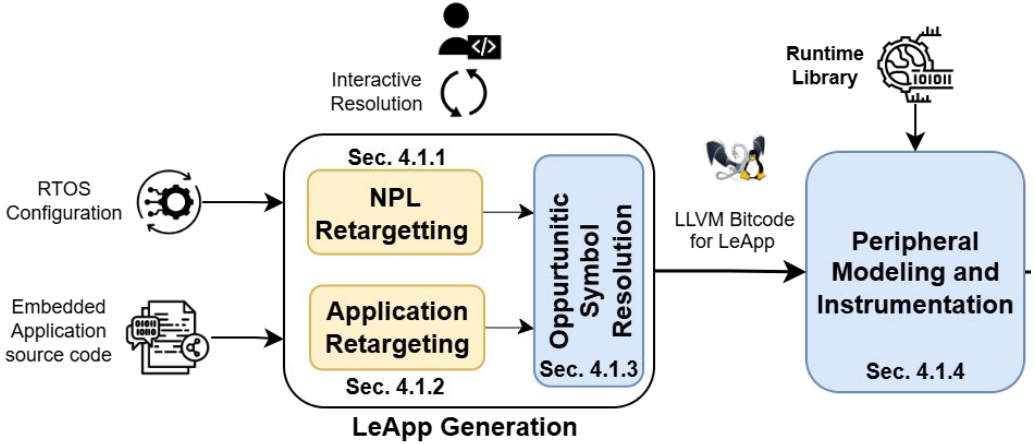
PURDUE UNIVERSITY

LEMIX : Framework Architecture



Automated POSIX-Swap integrates only compilable RTOS configs into the LPL build to ensure maximum compatibility.

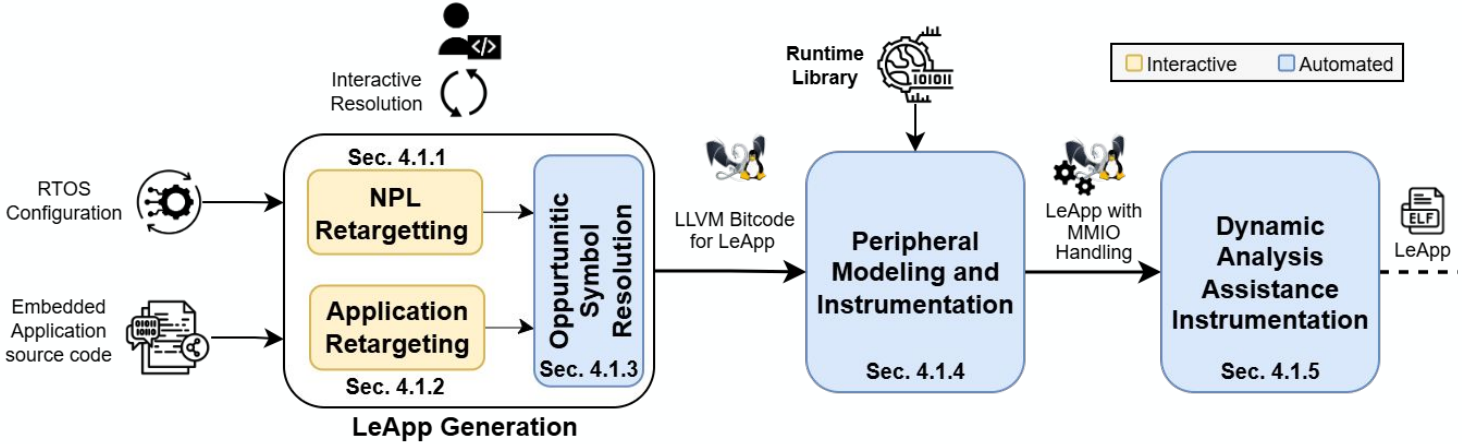
LEMIX : Framework Architecture




```
1   uint32_t HAL_RCC_GetSysClockFreq(void) {
2     uint32_t pllm = 0U, pllvco = 0U, pllp = 0U;
3     uint32_t sysclockfreq = 0U;
4     if (isMMIO(RCC->PLLCFGR)) {
5     pllm = get_input_from_stdin() & RCC_PLLCFGR_PLLM;
6     } else {
7     pllm = RCC->PLLCFGR & RCC_PLLCFGR_PLLM;
8     }
9     ...
10    sysclockfreq = pllvco / pllp; ...}
```

Listing 1: shows original MMIO accesses that are instrumented in . shows a div-by-zero bug in STM32.

LEMIX : Framework Architecture



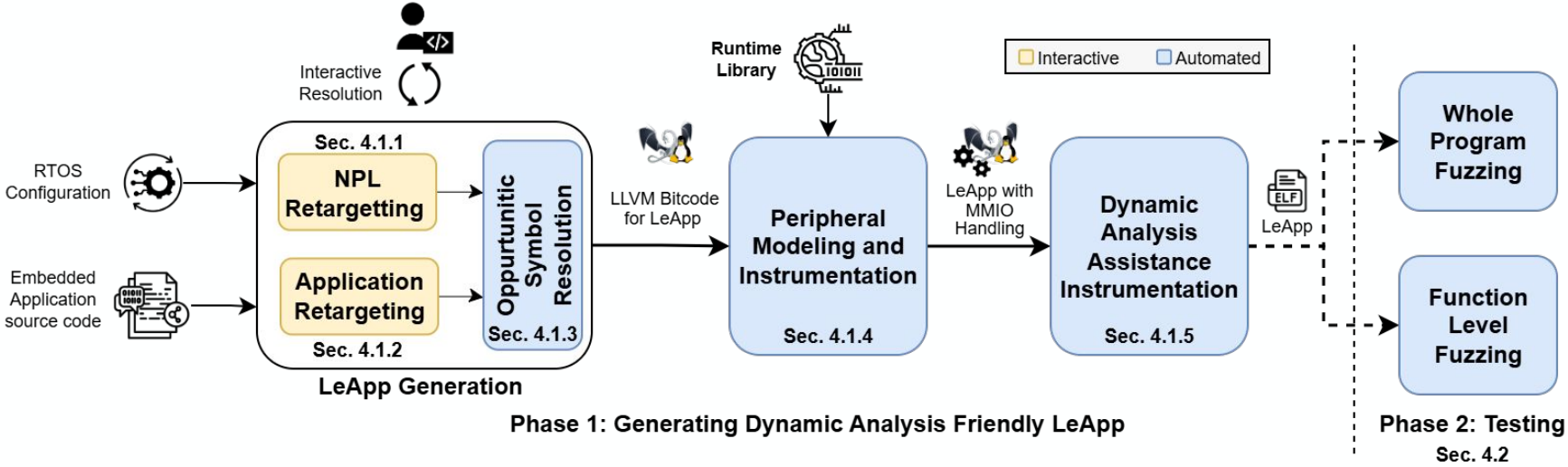
LEMIX : Framework Architecture

```
1   // Before Condition Weakening
2   while
3       ((NRF_CLOCK->LFCLKSTAT & CLOCK_LFCLKSTAT_STATE_Pos)🚫) {
4       // Busy Waiting
5       🔄 };
6   interesting_function();
7   // After Condition Weakening
8   label:
9       bool cond = (NRF_CLOCK->LFCLKSTAT &
10                   CLOCK_LFCLKSTAT_STATE_Pos); 🚫
11      new_cond = cond;
12      if (isMMIO(NRF_CLOCK->LFCLKSTAT) && stdin_read() % 2) 🔧
13          new_cond = !cond; 🔗
14      while (new_cond) {goto label};
15  interesting_function();
```

Listing 1: 🚫 indicates MMIO coverage blockers, 🔄 marks busy waiting due to unsolved constraints, 🔧 represents our instrumentation and 🔗 shows MMIO condition toggling.

PURDUE UNIVERSITY

LEMIX : Framework Architecture



Execution Control: Current Landscape

	System	Layer: Hardware	Layer: OS	Layer: Application	Layer: Function	System Type	Target: ISA(s)	Binary?	Availability[a]: Source	Availability[a]: Dataset
Pure Emulation	BaseSafe [54]	◪	◪	◪	🖳	2	ARM	✓	✓	✓
	Clements'21 [16]	🖳 ◪	◪	○	○	2	ARM	✓		
	Costin'16 [20]	🖳 🖳	◪	○	○	1	ARM, MIPS	✓		
	DICE [57]	🖳 🖳 ◪	○	○	○	2 3	ARM, MIPS	✓	✓	✓
	Firm-AFL [88]	🖳 🖳	◪	○	○	1	ARM, MIPS	✓	✓	✓
	Firmadyne [10]	🖳 🖳	◪	○ ◪	○	1	ARM, MIPS	✓	✓	
	FirmAE [46]	🖳 🖳	◪	○ ◪	○ ◪	1	ARM, MIPS	✓	✓	
	HALucinator [15]	🖳 ◪	◪	○	○ ◪	2 3	ARM	✓	✓	✓
	Li'21 [50]	◪	◪	○	○	2	N/A		~	~
	LuaQEMU [17]	🖳	—	○	○ ◪	3	ARM	✓	✓	
	P2IM [29]	🖳 🖳 ◪	○	○	○	2 3	ARM	✓	✓	✓
	PartEmu [37]	🖳 🖳 ◪	○	○ ◪	○	2	ARM	✓		
Hardware-in-the-Loop	Avatar2 [59]	🌐 🖳 🖳 ◪	—	🌐 ○	🌐 ○	3	ARM	✓	✓	✓
	Charm [77]	🌐 🖳 🖳	◪	○	○	1	ARM		✓	✓
	FEMU [49]	🌐 🖳 🖳	○	○	○	0 1 2 3	x86			
	Frankenstein [69]	🖳 🖳 ◪	◪	○	○ ◪	2	ARM	✓	✓	
	FirmCorn [35]	🖳 🖳	🌐 ◪	○	○ ◪	1	ARM, MIPS, x86	✓	✓	~
	Kammerstetter [42]	🌐 🖳 🖳	🌐 ◪	○	○	0 1	MIPS	✓		
	Pretender [36]	🌐 🖳 🖳 ◪	—	○	○	3	ARM	✓	✓	✓
	Prospect [43]	🌐 🖳 🖳	🌐 ◪	○	○	0 1	MIPS	✓		
	Surrogates [47]	🌐 🖳	○	○	○	0 1 2 3	ARM	✓		
Symbolic Abstractions	FIE [23]	*x*	—	○	○	3	MSP430	✓	✓	
	FirmUSB [38]	*x*	—	○	○	3	8051/52	✓		✓
	Firmalice [72]	*x*	○ *x*	○	○ *x*	1 2	ARM, PPC	✓		
	Laelaps [8]	*x*	○	○	○	2 3	ARM	✓	✓	~
Hybrid Approaches	Avatar [86]	🌐 🖳 *x*	🌐 ○	🌐 ○	🌐 ○ *x*	2 3	ARM	✓	✓	✓
	Inception [18]	🌐 🖳 *x*	○	○	○	2 3	ARM		✓	~
	Mousse [52]	🌐	🌐 ○	○	○ *x*	1	ARM		✓	~

[a]Partial availability is indicated via ~.

SoK Enabling Security Analysis: https://dl.acm.org/doi/pdf/10.1145/3433210.3453093